

PROJECT TECHNICAL GUIDE

GMRID Hybrid Risk Classification

Complete workflow, routing logic, presentation narrative, and exact cell-by-cell code explanation from data preparation through Stages A, B, and C.

Scope

4 notebooks / 113 code cells

Source snapshot

2026-09-12

Pipeline

Rules -> Embeddings -> RAG + LLM

Output

Technical and presentation-ready guide

How to use this guide

Start with the workflow and presentation sections for a concise project explanation. Use the notebook sections as a complete code reference: every code cell is reproduced exactly and followed by an operation-level explanation.

1. End-to-end workflow

Inputs, taxonomy, outputs, and system architecture.

2. Routing procedure

Exact acceptance and escalation decisions between stages.

3. Presentation narrative

A clear talk track, metrics, and likely questions.

4. Current implementation notes

Execution dependencies, stale outputs, and the Stage C ordering issue.

5. Data preparation

38 code cells with explanations and source.

6. Stage A

28 code cells with explanations and source.

7. Stage B

28 code cells with explanations and source.

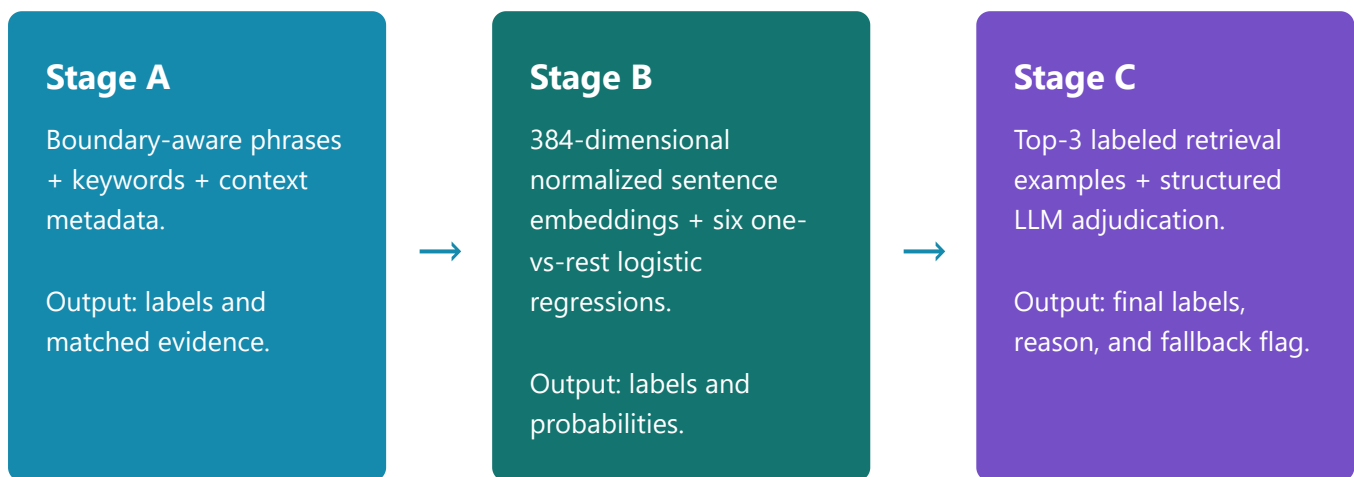
8. Stage C

19 code cells with explanations and source.

Reading convention. “Exact notebook code” is taken directly from the saved `.ipynb` source. Lines may wrap visually to fit A4 pages, but no code text is rewritten. Metrics are explicitly labeled as recorded notebook output or current exported-artifact counts.

End-to-end workflow

The project is a cascading multi-label classifier for maritime and logistics risk events. It first resolves obvious cases with transparent rules, then uses semantic learning to cover language variation, and finally reserves an LLM for ambiguous disagreements. The design controls cost and latency by sending only difficult records to Stage C.



Preparation layer

1. Load GMRID train/test splits and inspect schema, missingness, vocabulary, and class distribution.
2. Map source `Category` and `Summarized_label` values into six ordered multi-label targets.
3. Keep only events covered by the target taxonomy: current files contain **3,322 train** and **819 test** rows.
4. Create `input_text` for semantic/LLM processing and lowercase whitespace-normalized `normalized_text` for deterministic matching.
5. Export POC CSVs; ground-truth source fields remain available for evaluation but are excluded from classifier features.

Six target risks

Risk ID	Meaning	Typical evidence
weather_disruption	Severe-weather disruption	storms, flooding, winds, cyclone, hurricane
natural_disaster	Geological/natural hazards	earthquake, tsunami, volcano, landslide
port_operational_disruption	Reduced port or cargo efficiency	congestion, delays, backlogs, capacity constraints
port_closure	Full or partial closure/suspension	port closed, terminal closure, suspended operations
labor_strike_disruption	Worker or union action	strike, industrial action, walkout, labor dispute
maritime_security_navigation_disruption	Security or navigational restriction	piracy, advisory, channel closure, vessel traffic halt

Artifacts passed between stages

Producer	Artifact	Purpose
Preparation	data/train_poc.csv, data/test_poc.csv	Text, truth, IDs, and selected metadata.
Stage A	outputs/stage_a_*_predictions.csv	ID and JSON list of deterministic labels.
Stage B	outputs/stage_b_embeddings.npz	Ordered IDs, normalized float32 embeddings, and model name.
Stage B	outputs/stage_b_*_predictions.csv	Semantic labels and per-risk probabilities.
Stage B	outputs/stage_c_*_input.csv	Unified routing and evidence handoff to Stage C.
Stage C	stage_c_train_sample_predictions.csv, stage_c_test_predictions.csv, stage_c_metrics.csv	Adjudication audit trail, final labels, and metrics. These files are not present in the current workspace snapshot.

Complete routing procedure

Stage A scoring

Each risk is scored independently. A distinct strong phrase contributes 3 points, a distinct keyword not already contained in a matched phrase contributes 1, and relevant port/maritime context contributes 1 only after textual evidence exists. Predictions are labels whose scores meet their per-risk threshold: 3 for every class except port closure, which uses 4.

Word-boundary regexes prevent accidental substring matches. Longer phrases are checked first, nested evidence is not double-counted, and matched evidence is retained for explainability.

Stage B learning

Every record is embedded by all-MiniLM-L6-v2. Six balanced logistic regressions are trained in one-vs-rest form. Three-fold out-of-fold training probabilities select an F1-maximizing threshold for each risk; the final full-training model produces test probabilities.

A/B decision table

Stage A	Stage B	Decision	Reason code
Non-empty	Exactly the same non-empty set	Accept Stage A labels; do not call Stage C.	stage_a_stage_b_agree
Empty	Non-empty	Accept Stage B labels; do not call Stage C.	stage_a_empty_stage_b_confident
Empty	Empty	Route to Stage C.	both_stages_empty
Non-empty	Empty	Route to Stage C.	stage_b_empty
Different sets	Different sets	Route to Stage C.	predictions_disagree

Current exported routing counts

Split	Agreement	A empty / B confident	Disagree	Both empty	B empty	Total routed
Train (3,322)	1,766	781	720	40	15	775
Test (819)	464	161	179	11	4	194

Stage C adjudication

1. Select only routed rows. The training review sample caps three reason groups at 45, 40, and 15 cases; all 194 routed test rows are included.
2. Retrieve the top three training examples by dot-product similarity over normalized embeddings. Train evaluation excludes the query itself.
3. Send target text, A/B labels, B probabilities, route reason, and retrieved labeled examples to the LLM.
4. Require strict JSON containing only an allowed list of risk IDs and a brief reason.
5. Retry once after an exception. If both attempts fail, fall back to Stage B when available, otherwise Stage A, otherwise an empty list.
6. Merge adjudicated rows back into the complete test set. Routed rows use Stage C output; all other rows use the earlier accepted prediction.

How to explain the project

- 1 Problem.** "Maritime risk events are multi-label and expressed in varied language. One event can simultaneously describe weather, port disruption, and closure, so a single-label classifier is insufficient."
- 2 Design idea.** "I use a cascade: cheap, explainable rules first; semantic machine learning second; and an LLM only for unresolved ambiguity. This gives traceability while controlling API cost."
- 3 Data.** "I map GMRID categories into six business-facing risks, preserve multi-label truth, create raw and normalized text, and retain 3,322 training plus 819 test records covered by the taxonomy."
- 4 Stage A.** "Rules score boundary-safe phrases, keywords, and context. Every predicted label includes its evidence. High precision makes this a strong first gate, while uncertain empty results continue."
- 5 Stage B.** "A pretrained sentence transformer converts each event into a 384-dimensional normalized vector. Six balanced logistic regressions learn independent risks, and out-of-fold F1 chooses a threshold for each class."
- 6 Routing.** "Exact A/B agreement is accepted. When A is empty but B is confident, B is accepted. All disagreements or empty outcomes are escalated. In the current test handoff, 194 of 819 rows - 23.7% - need Stage C."
- 7 Stage C.** "For each escalated case, I retrieve three semantically similar labeled training events. The LLM receives these examples plus earlier predictions and probabilities, then returns schema-constrained labels and a short rationale."
- 8 Reliability.** "Every merge is ID-based, predictions are serialized as JSON, retrieval IDs and reasons are exported for audit, and an exception fallback avoids losing a classification when the API fails."

Recorded model results

0.9141 Stage A test micro precision	0.7634 Stage A test micro F1	0.6311 Stage B test micro F1	23.7% Test rows routed to Stage C
--	---	---	--

Stage A values come from the saved Stage A notebook output. Stage B values come from the saved Stage B evaluation output. No completed Stage C metrics are recorded in the current notebook/output snapshot, so no final-pipeline score is claimed.

Questions you may be asked

Question	Answer to give
Why not use the LLM for every row?	The cascade reduces cost and latency, preserves deterministic evidence for clear cases, and limits the LLM to ambiguity.
Why multi-label?	A single event can have both a cause and an operational consequence, such as weather plus port closure.
Why normalized embeddings?	With L2-normalized vectors, the dot product equals cosine similarity, making retrieval efficient and interpretable.
Why one model per risk?	One-vs-rest supports independent, simultaneous labels and per-class decision thresholds.
How is leakage controlled?	Ground-truth source fields are excluded from classifier text; training threshold estimates use out-of-fold probabilities; test evaluation is held out; self-retrieval is disabled for train examples.
What is auditable?	Stage A matched evidence, Stage B probabilities, route reasons, retrieved IDs/examples, LLM reason, and fallback status.

Current-state notes

Stage C training-order issue. The notebook defines `run_adjudication` but evaluates `train_llm_input["stage_c_predictions"]` before calling `train_llm_input = run_adjudication(train_llm_input)`. As saved, Cell 15 cannot complete in a clean top-to-bottom run. This report documents the issue only; no notebook was changed.

Stage C is not complete in the saved snapshot. Cells 10 and 15-19 have no recorded execution counts/results, and the expected Stage C output CSVs are absent. Final-pipeline and routed-only metrics therefore remain unverified.

Some Stage B outputs are stale relative to their cells. Several execution counts are blank, and visible output attached near the model-loading/embedding cells describes shapes that do not match the current source cell's print statements. Treat the source code as authoritative and re-run cleanly before reporting production results.

Runtime requirements. Stage B deliberately requires CUDA and downloads `all-MiniLM-L6-v2`. Stage C installs the OpenAI package, needs a valid API key and network access, and can incur API cost.

Model-validation considerations. Stage B uses ordinary shuffled KFold, not a multi-label-stratified splitter. Thresholds are selected from the same out-of-fold predictions used for reported training metrics. For a stronger final study, freeze thresholds using a dedicated validation design and run the untouched test set once.

Artifact/status distinction. Current routing counts were read from exported Stage C input CSVs. Performance figures are recorded notebook outputs, not newly executed results; the user's notebooks were left unchanged.

Data Preparation

Dataset loading, taxonomy mapping, POC filtering, text construction, and CSV export

SOURCE notebooks/01_data_preparation.ipynb

38 CODE CELLS

CELL 1

Code cell 1

Execution count: 55

WHAT THIS CELL IMPLEMENTS

Imports pandas for tabular processing, NumPy for numerical work, Matplotlib for plotting, and Hugging Face Datasets for downloading GMRID. These libraries form the preparation and exploratory-analysis toolset.

EXACT NOTEBOOK CODE

```
1 | import pandas as pd
2 | import numpy as np
3 | import matplotlib.pyplot as plt
4 |
5 | from datasets import load_dataset
```

CELL 2

Code cell 2

Execution count: 56

WHAT THIS CELL IMPLEMENTS

Downloads and loads the `windchimeran/GMRID` dataset through `load\_dataset`. The returned DatasetDict is retained in memory for split-level access.

EXACT NOTEBOOK CODE

```
1 | dataset = load_dataset("windchimeran/GMRID")
```

CELL 3**Code cell 3**

Execution count: 57

WHAT THIS CELL IMPLEMENTS

Prints the DatasetDict representation so the available splits, feature schema, and row counts can be inspected.

EXACT NOTEBOOK CODE

```
1 | print(dataset)
```

CELL 4**Code cell 4**

Execution count: 58

WHAT THIS CELL IMPLEMENTS

Prints the dataset split keys. This confirms which named partitions can be selected in the following cell.

EXACT NOTEBOOK CODE

```
1 | print(dataset.keys())
```

CELL 5**Code cell 5**

Execution count: 59

WHAT THIS CELL IMPLEMENTS

Converts the Hugging Face `train` and `test` splits into pandas DataFrames named `train\_df` and `test\_df`.

EXACT NOTEBOOK CODE

```
1 | train_df = dataset["train"].to_pandas()
2 | test_df = dataset["test"].to_pandas()
```

CELL 6**Code cell 6**

Execution count: 60

WHAT THIS CELL IMPLEMENTS

Prints both DataFrame shapes and the complete training-column list. This is an early schema and volume check.

EXACT NOTEBOOK CODE

```
1 | print(train_df.shape)
2 | print(test_df.shape)
3 | print(train_df.columns.tolist())
```

CELL 7

Code cell 7

Execution count: 61

WHAT THIS CELL IMPLEMENTS

Displays the first training rows for a visual inspection of values, formatting, and candidate features.

EXACT NOTEBOOK CODE

```
1 | train_df.head()
```

CELL 8

Code cell 8

Execution count: 62

WHAT THIS CELL IMPLEMENTS

Calls `DataFrame.info()` to report column data types, non-null counts, and memory use. Despite the comment saying 'missing columns,' the operation checks missing values within columns rather than missing column names.

EXACT NOTEBOOK CODE

```
1 | # checking missing columns
2 | train_df.info()
```

CELL 9

Code cell 9

Execution count: 63

WHAT THIS CELL IMPLEMENTS

Loops over every training column and prints its distinct-value count. This separates low-cardinality categorical fields from IDs, dates, text, and continuous attributes.

EXACT NOTEBOOK CODE

```
1 | # unique count for each column
2 | for col in train_df.columns:
3 |     print(col, train_df[col].nunique())
```

CELL 11**Code cell 11**

Execution count: 64

WHAT THIS CELL IMPLEMENTS

Counts occurrences of each `Summarized\_label`, ordered by frequency. The result reveals the high-level causal-label distribution.

EXACT NOTEBOOK CODE

```
1 | train_df["Summarized_label"].value_counts()
```

CELL 12**Code cell 12**

Execution count: 65

WHAT THIS CELL IMPLEMENTS

Counts `Category` values and shows the 50 most frequent combinations. Because the source can contain comma-separated categories, these are raw category strings rather than individual exploded labels.

EXACT NOTEBOOK CODE

```
1 | train_df["Category"].value_counts().head(50)
```

CELL 13**Code cell 13**

Execution count: 66

WHAT THIS CELL IMPLEMENTS

Drops missing summarized labels, extracts unique values, sorts them, and displays the full label vocabulary.

EXACT NOTEBOOK CODE

```
1 | sorted(train_df["Summarized_label"].dropna().unique())
```

CELL 14**Code cell 14**

Execution count: 67

WHAT THIS CELL IMPLEMENTS

Drops missing categories, extracts and sorts unique raw category strings, and displays the first 100 values for taxonomy design.

EXACT NOTEBOOK CODE

```
1 | sorted(train_df["Category"].dropna().unique())[:100]
```

CELL 16

Code cell 16

Execution count: 68

WHAT THIS CELL IMPLEMENTS

Defines the six-label target risk taxonomy. Each stable machine-readable ID is paired with a presentation name and a natural-language scope description used consistently across later stages.

EXACT NOTEBOOK CODE

```
1 | RISK_TAXONOMY = {
2 |     "weather_disruption": {
3 |         "name": "Weather Disruption",
4 |         "description": (
5 |             "Disruption to trade, transport, ports, logistics, or infrastructure "
6 |             "caused by severe weather such as storms, flooding, high winds, "
7 |             "cyclones, or similar weather-related hazards."
8 |         )
9 |     },
10 |
11 |     "natural_disaster": {
12 |         "name": "Natural Disaster",
13 |         "description": (
14 |             "Disruption caused by natural disasters such as earthquakes, "
15 |             "tsunamis, volcanic activity, landslides, or other geological hazards."
16 |         )
17 |     },
18 |
19 |     "port_operational_disruption": {
20 |         "name": "Port Operational Disruption",
21 |         "description": (
22 |             "Disruption to normal port or cargo operations caused by congestion, "
23 |             "capacity limitations, operational delays, cargo disruption, "
24 |             "or reduced terminal efficiency."
25 |         )
26 |     },
27 |
28 |     "port_closure": {
```

```

29 |         "name": "Port Closure",
30 |         "description": (
31 |             "Full or partial closure, suspension, or shutdown of a port, terminal, "
32 |             "pier, berth, or related maritime facility."
33 |         )
34 |     },
35 |
36 |     "labor_strike_disruption": {
37 |         "name": "Labor / Strike Disruption",
38 |         "description": (
39 |             "Disruption caused by worker strikes, industrial action, labor disputes, "
40 |             "walkouts, or related workforce actions affecting transport or logistics."
41 |         )
42 |     },
43 |
44 |     "maritime_security_navigation_disruption": {
45 |         "name": "Maritime Security / Navigation Disruption",
46 |         "description": (
47 |             "Disruption or increased operational risk to maritime transport caused "
48 |             "by piracy, maritime security threats, navigation restrictions, "
49 |             "maritime advisories, or waterway disruption."
50 |         )
51 |     }
52 | }

```

CELL 17

Code cell 17

Execution count: 69

WHAT THIS CELL IMPLEMENTS

Iterates through the taxonomy and prints each internal risk ID with its human-readable name, verifying the order and definitions.

EXACT NOTEBOOK CODE

```

1 | for risk_id, data in RISK_TAXONOMY.items():
2 |     print(risk_id, "->", data["name"])

```

WHAT THIS CELL IMPLEMENTS

Defines source-category groups for each target risk. These sets are the deterministic bridge from GMRID's original category vocabulary to the project's six-class multi-label taxonomy.

EXACT NOTEBOOK CODE

```
1 | # Group the source categories that belong to each target risk.
2 | #
3 | # Summarized_label is also used below for causal labels such as Weather
4 | # and Worker Strike. Category continues to provide operational event labels
5 | # such as Port Closure and Port Congestion.
6 | CATEGORY_GROUPS = {
7 |
8 |     "weather_disruption": {
9 |         "Weather Advisory",
10 |        "Flooding",
11 |        "Storm",
12 |        "Tropical Cyclone / Storm",
13 |        "Severe Winds",
14 |        "Typhoon",
15 |        "Hurricane",
16 |        "Hail",
17 |        "Tornado",
18 |        "Blizzard"
19 |    },
20 |
21 |    "natural_disaster": {
22 |        "Earthquake",
23 |        "Landslide"
24 |    },
25 |
26 |    "port_operational_disruption": {
27 |        "Port Congestion",
28 |        "Port Disruption",
29 |        "Cargo Disruption"
30 |    },
31 |
32 |    "port_closure": {
```

```

33 |         "Port Closure"
34 |     },
35 |
36 |     "labor_strike_disruption": {
37 |         "Port Strike",
38 |         "Cargo Transportation Strike",
39 |         "Industrial Action",
40 |         "General Strike",
41 |         "Miscellaneous Strikes",
42 |         "Civil Service Strike"
43 |     },
44 |
45 |     "maritime_security_navigation_disruption": {
46 |         "Maritime Advisory",
47 |         "Piracy",
48 |         "Waterway Closure / Disruption"
49 |     }
50 | }

```

CELL 20

Code cell 20

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Defines row-level ground-truth mapping. It splits comma-separated categories, handles missing values, assigns weather and labor using both `Category` and `Summarized\_label`, assigns the remaining risks through set intersections, preserves taxonomy order, and permits multiple labels per event.

EXACT NOTEBOOK CODE

```

1 |
2 |
3 | def map_row_to_risks(row):
4 |
5 |     category_text = row.get("Category", "")
6 |     summarized_label = row.get("Summarized_label", "")
7 |
8 |     if pd.isna(category_text):
9 |         categories = set()
10 |     else:
11 |         categories = {

```

```

12 |         category.strip()
13 |         for category in str(category_text).split(",")
14 |             if category.strip()
15 |         }
16 |
17 |     if pd.isna(summarized_label):
18 |         summarized_label = ""
19 |     else:
20 |         summarized_label = str(summarized_label).strip()
21 |
22 |     mapped_risks = []
23 |
24 |     natural_categories = CATEGORY_GROUPS[
25 |         "natural_disaster"
26 |     ]
27 |
28 |     weather_categories = CATEGORY_GROUPS[
29 |         "weather_disruption"
30 |     ]
31 |
32 |     # Summarized_label captures the cause of many port disruptions.
33 |     #
34 |     # A standalone earthquake or landslide is kept under natural_disaster
35 |     # instead of also being automatically labeled as weather_disruption.
36 |     # If an event explicitly contains a weather category such as Flooding,
37 |     # it can still receive both labels.
38 |     has_weather_risk = (
39 |         (
40 |             summarized_label == "Weather"
41 |             and not categories.intersection(natural_categories)
42 |         )
43 |         or bool(categories.intersection(weather_categories))
44 |     )
45 |
46 |     if has_weather_risk:
47 |         mapped_risks.append(
48 |             "weather_disruption"
49 |         )
50 |

```

```

51 |     if categories.intersection(natural_categories):
52 |         mapped_risks.append(
53 |             "natural_disaster"
54 |         )
55 |
56 |     if categories.intersection(
57 |         CATEGORY_GROUPS["port_operational_disruption"]
58 |     ):
59 |         mapped_risks.append(
60 |             "port_operational_disruption"
61 |         )
62 |
63 |     if categories.intersection(
64 |         CATEGORY_GROUPS["port_closure"]
65 |     ):
66 |         mapped_risks.append(
67 |             "port_closure"
68 |         )
69 |
70 |     # Worker Strike is a causal source label and catches port disruptions
71 |     # caused by labor action even when Category only says Port Disruption.
72 |     has_labor_risk = (
73 |         summarized_label == "Worker Strike"
74 |         or bool(
75 |             categories.intersection(
76 |                 CATEGORY_GROUPS["labor_strike_disruption"]
77 |             )
78 |         )
79 |     )
80 |
81 |     if has_labor_risk:
82 |         mapped_risks.append(
83 |             "labor_strike_disruption"
84 |         )
85 |
86 |     if categories.intersection(
87 |         CATEGORY_GROUPS[
88 |             "maritime_security_navigation_disruption"
89 |         ]

```

```
90 | ):
91 |     mapped_risks.append(
92 |         "maritime_security_navigation_disruption"
93 |     )
94 |
95 |     # Labels are appended in taxonomy order, so no set conversion is needed.
96 |     return mapped_risks
```

CELL 21

Code cell 21

Execution count: 72

WHAT THIS CELL IMPLEMENTS

Applies the row-level mapper to every train and test record to create `true\_risks`. The comments explicitly identify `Category` and `Summarized\_label` as ground-truth-only fields that must not become classifier inputs.

EXACT NOTEBOOK CODE

```
1 | # # apply the function to create "true_risk"
2 | # train_df["true_risks"] = train_df["Category"].apply(
3 | #     map_categories_to_risks
4 | # )
5 |
6 | # test_df["true_risks"] = test_df["Category"].apply(
7 | #     map_categories_to_risks
8 | #)
9 |
10 | # Apply the row-level mapping to create deterministic multi-label targets.
11 | #
12 | # Category and Summarized_label are ground-truth sources only.
13 | # They must not be passed to the Stage A classifier as input features.
14 | train_df["true_risks"] = train_df.apply(
15 |     map_row_to_risks,
16 |     axis=1
17 | )
18 |
19 | test_df["true_risks"] = test_df.apply(
20 |     map_row_to_risks,
21 |     axis=1
22 | )
```

CELL 22

Code cell 22

Execution count: 73

WHAT THIS CELL IMPLEMENTS

Displays 20 category-to-target mappings for a manual sanity check of the newly created labels.

EXACT NOTEBOOK CODE

```
1 | train_df[
2 |     ["Category", "true_risks"]
3 | ].head(20)
```

CELL 23**Code cell 23**

Execution count: 74

WHAT THIS CELL IMPLEMENTS

Creates `has\_target\_risk`, a Boolean indicating whether each event maps to at least one of the six target risks. The comment says 'remove rows,' but this cell only creates the flag; filtering happens later.

EXACT NOTEBOOK CODE

```
1 | # remove rows where true_risk is empty
2 | train_df["has_target_risk"] = train_df["true_risks"].apply(
3 |     lambda x: len(x) > 0
4 | )
5 |
6 | test_df["has_target_risk"] = test_df["true_risks"].apply(
7 |     lambda x: len(x) > 0
8 | )
```

CELL 24**Code cell 24**

Execution count: 75

WHAT THIS CELL IMPLEMENTS

Prints the number of covered records in train and test relative to each split's total size.

EXACT NOTEBOOK CODE

```
1 | # dataset covered by risk taxonomy
2 | print(
3 |     "Train covered:",
4 |     train_df["has_target_risk"].sum(),
5 |     "/",
6 |     len(train_df)
7 | )
8 |
9 | print(
10 |     "Test covered:",
11 |     test_df["has_target_risk"].sum(),
12 |     "/",
13 |     len(test_df)
14 | )
```

CELL 25**Code cell 25**

Execution count: 76

WHAT THIS CELL IMPLEMENTS

Converts the coverage flags to percentages by taking their means, multiplying by 100, and rounding to two decimals.

EXACT NOTEBOOK CODE

```
1 | # percentage dataset covered by risk taxonomy
2 | print(
3 |     "Train coverage:",
4 |     round(train_df["has_target_risk"].mean() * 100, 2),
5 |     "%"
6 | )
7 |
8 | print(
9 |     "Test coverage:",
10 |    round(test_df["has_target_risk"].mean() * 100, 2),
11 |    "%"
12 | )
```

CELL 26**Code cell 26**

Execution count: 77

WHAT THIS CELL IMPLEMENTS

Filters both splits to rows with at least one target risk and uses `.copy()` to produce independent proof-of-concept DataFrames.

EXACT NOTEBOOK CODE

```
1 | # Create POC datasets , filter to records that contain at least one of your six risks
2 |
3 | train_poc = train_df[
4 |     train_df["has_target_risk"]
5 | ].copy()
6 |
7 | test_poc = test_df[
8 |     test_df["has_target_risk"]
9 | ].copy()
```

CELL 27**Code cell 27**

Execution count: 78

WHAT THIS CELL IMPLEMENTS

Resets the filtered DataFrame indices to consecutive integers while discarding the old index.

EXACT NOTEBOOK CODE

```
1 | # Reset indices
2 | train_poc.reset_index(drop=True, inplace=True)
3 | test_poc.reset_index(drop=True, inplace=True)
```

CELL 28**Code cell 28**

Execution count: 79

WHAT THIS CELL IMPLEMENTS

Prints the final POC train and test shapes after taxonomy filtering.

EXACT NOTEBOOK CODE

```
1 | print(train_poc.shape)
2 | print(test_poc.shape)
```

CELL 29**Code cell 29**

Execution count: 80

WHAT THIS CELL IMPLEMENTS

Displays the first prepared training rows to inspect the filtered schema and mapped labels.

EXACT NOTEBOOK CODE

```
1 | train_poc.head()
```

CELL 31**Code cell 31**

Execution count: 81

WHAT THIS CELL IMPLEMENTS

Explodes each training row's list of true risks into one label per row, then counts label frequencies. Multi-label events therefore contribute once to every assigned class.

EXACT NOTEBOOK CODE

```
1 | # checking class distribution for training data
2 | train_label_counts = (
3 |     train_poc["true_risks"]
4 |     .explode()
5 |     .value_counts()
6 | )
7 |
8 | train_label_counts
```

CELL 32

Code cell 32

Execution count: 82

WHAT THIS CELL IMPLEMENTS

Repeats the exploded label-frequency calculation on the test POC split.

EXACT NOTEBOOK CODE

```
1 | # checking class distribution for test data
2 | test_label_counts = (
3 |     test_poc["true_risks"]
4 |     .explode()
5 |     .value_counts()
6 | )
7 |
8 | test_label_counts
```

CELL 33

Code cell 33

Execution count: 83

WHAT THIS CELL IMPLEMENTS

Draws a bar chart of training-class counts and labels both axes. This visualizes the natural class imbalance retained by the rule-based Stage A design.

EXACT NOTEBOOK CODE

```
1 | # count plot for class distribution
2 | train_label_counts.plot(
3 |     kind="bar",
4 |     title="Training Risk Distribution",
5 |     figsize=(10, 5)
6 | )
7 |
8 | plt.ylabel("Number of records")
9 | plt.xlabel("Risk")
10 | plt.show()
```

CELL 35

Code cell 35

Execution count: 84

WHAT THIS CELL IMPLEMENTS

Defines `prepare\_text`. It prefers the existing `Headline\_Details` field, falls back to concatenated headline and details when that field is missing, safely handles nulls, converts to text, and trims outer whitespace.

EXACT NOTEBOOK CODE

```
1 | # Create your classifier input text
2 | def prepare_text(row):
3 |     text = row["Headline_Details"]
4 |
5 |     if pd.isna(text):
6 |         headline = "" if pd.isna(row["Headline"]) else str(row["Headline"])
7 |         details = "" if pd.isna(row["Details"]) else str(row["Details"])
8 |
9 |         text = headline + " " + details
10 |
11 |     return str(text).strip()
```

CELL 36

Code cell 36

Execution count: 85

WHAT THIS CELL IMPLEMENTS

Applies `prepare\_text` row-wise to train and test, producing `input\_text` for embeddings, LLM prompts, and human inspection.

EXACT NOTEBOOK CODE

```
1 | # applying the function to the data
2 | train_poc["input_text"] = train_poc.apply(
3 |     prepare_text,
4 |     axis=1
5 | )
6 |
7 | test_poc["input_text"] = test_poc.apply(
8 |     prepare_text,
9 |     axis=1
10 | )
```

CELL 37

Code cell 37

Execution count: 86

WHAT THIS CELL IMPLEMENTS

Displays the head of the training POC after adding `input\_text`.

EXACT NOTEBOOK CODE

```
1 | train_poc.head()
```

CELL 38

Code cell 38

Execution count: 87

WHAT THIS CELL IMPLEMENTS

Imports regular expressions and defines light normalization: string conversion, lowercase transformation, collapsing all whitespace runs to one space, and trimming.

EXACT NOTEBOOK CODE

```
1 | import re
2 |
3 | def normalize_text(text):
4 |     text = str(text).lower()
5 |
6 |     text = re.sub(
7 |         r"\s+",
8 |         " ",
9 |         text
10 |    )
11 |
12 |    return text.strip()
```

CELL 39

Code cell 39

Execution count: 88

WHAT THIS CELL IMPLEMENTS

Applies text normalization to both splits and stores the result in `normalized\_text`, the Stage A matching field.

EXACT NOTEBOOK CODE

```
1 | train_poc["normalized_text"] = train_poc[
2 |     "input_text"
3 | ].apply(normalize_text)
4 |
5 | test_poc["normalized_text"] = test_poc[
6 |     "input_text"
7 | ].apply(normalize_text)
```

CELL 40

Code cell 40

Execution count: 89

WHAT THIS CELL IMPLEMENTS

Randomly samples 10 training records with a fixed seed and displays their headline, source category, target risks, and normalized text for reproducible quality checking.

EXACT NOTEBOOK CODE

```
1 | train_poc[
2 |     [
3 |         "Headline",
4 |         "Category",
5 |         "true_risks",
6 |         "normalized_text"
7 |     ]
8 | ].sample(10, random_state=42)
```

CELL 41

Code cell 41

Execution count: 90

WHAT THIS CELL IMPLEMENTS

Shows the first prepared rows again after the text fields have been created.

EXACT NOTEBOOK CODE

```
1 | train_poc.head()
```

CELL 42

Code cell 42

Execution count: 91

WHAT THIS CELL IMPLEMENTS

Selects and orders the final handoff columns. It retains identifiers, source text and metadata, ground truth, and both raw and normalized classifier text while dropping unrelated source fields.

EXACT NOTEBOOK CODE

```
1 | columns_to_keep = [  
2 |     "id",  
3 |     "Headline",  
4 |     "Details",  
5 |     "Severity",  
6 |     "Region",  
7 |     "Datetime",  
8 |     "lat",  
9 |     "lon",  
10 |     "maritime_label",  
11 |     "found_ports",  
12 |     "contains_port_info",  
13 |     "Category",  
14 |     "Summarized_label",  
15 |     "true_risks",  
16 |     "input_text",  
17 |     "normalized_text"  
18 | ]  
19 |  
20 | train_poc = train_poc[columns_to_keep].copy()  
21 | test_poc = test_poc[columns_to_keep].copy()
```

CELL 43

Code cell 43

Execution count: 92

WHAT THIS CELL IMPLEMENTS

Creates the project data directory when needed and exports `train\_poc.csv` and `test\_poc.csv` without pandas index columns. These files are the stable inputs to Stages A and B.

EXACT NOTEBOOK CODE

```
1 | from pathlib import Path  
2 |  
3 | DATA_DIR = Path("../data")  
4 | DATA_DIR.mkdir(exist_ok=True)  
5 |  
6 | train_poc.to_csv(DATA_DIR / "train_poc.csv", index=False)  
7 | test_poc.to_csv(DATA_DIR / "test_poc.csv", index=False)
```

Stage A - Deterministic Rules

High-precision phrase, keyword, and context scoring

SOURCE notebooks/02_stage_a_rules.ipynb

28 CODE CELLS

CELL 2

Code cell 2

Execution count: 2

WHAT THIS CELL IMPLEMENTS

Imports tabular, numerical, literal-parsing, regular-expression, and path utilities required by the rule pipeline.

EXACT NOTEBOOK CODE

```
1 | import pandas as pd
2 | import numpy as np
3 | import ast
4 | import re
5 | from pathlib import Path
```

CELL 3

Code cell 3

Execution count: 3

WHAT THIS CELL IMPLEMENTS

Loads the prepared train and test POC CSVs from `../data` and prints their shapes to verify the handoff from data preparation.

EXACT NOTEBOOK CODE

```
1 | DATA_DIR = Path("../data")
2 |
3 | train_poc = pd.read_csv(DATA_DIR / "train_poc.csv")
4 | test_poc = pd.read_csv(DATA_DIR / "test_poc.csv")
5 |
6 | print("Train shape:", train_poc.shape)
7 | print("Test shape:", test_poc.shape)
```

CELL 4**Code cell 4**

Execution count: 4

WHAT THIS CELL IMPLEMENTS

Defines a safe list-restoration helper and applies it to `true\_risks`. CSV serialization turned Python lists into strings, so `ast.literal\_eval` reconstructs them without executing arbitrary code.

EXACT NOTEBOOK CODE

```
1 | # CSV does not preserve Python lists or booleans reliably
2 | def restore_list(value):
3 |     if isinstance(value, list):
4 |         return value
5 |
6 |     if pd.isna(value):
7 |         return []
8 |
9 |     return ast.literal_eval(value)
10 |
11 | train_poc["true_risks"] = train_poc["true_risks"].apply(restore_list)
12 | test_poc["true_risks"] = test_poc["true_risks"].apply(restore_list)
```

CELL 5**Code cell 5**

Execution count: 5

WHAT THIS CELL IMPLEMENTS

Prints the runtime type and value of the first restored target. This confirms that labels are lists rather than raw CSV strings.

EXACT NOTEBOOK CODE

```
1 | print(type(train_poc.loc[0, "true_risks"]))
2 | print(train_poc.loc[0, "true_risks"])
```

CELL 6**Code cell 6**

Execution count: 6

WHAT THIS CELL IMPLEMENTS

Defines permissive Boolean normalization and applies it to available maritime and port-information columns in both splits. Accepted truthy forms are Boolean true, `true`, `1`, and `yes` after string normalization.

EXACT NOTEBOOK CODE

```
1 | # Normalize metadata booleans
2 | def to_bool(value):
3 |     if isinstance(value, bool):
4 |         return value
5 |
6 |     return str(value).strip().lower() in {
7 |         "true", "1", "yes"
8 |     }
9 |
10 |
11 | for col in ["maritime_label", "contains_port_info"]:
12 |     if col in train_poc.columns:
13 |         train_poc[col] = train_poc[col].apply(to_bool)
14 |
15 |     if col in test_poc.columns:
16 |         test_poc[col] = test_poc[col].apply(to_bool)
```

CELL 7**Code cell 7**

Execution count: 7

WHAT THIS CELL IMPLEMENTS

Prints all loaded column names to confirm the Stage A input schema.

EXACT NOTEBOOK CODE

```
1 | print(train_poc.columns.tolist())
```

CELL 8**Code cell 8**

Execution count: 8

WHAT THIS CELL IMPLEMENTS

Displays headline, original category, ground-truth risks, and normalized text for the first rows. `Category` is shown only for inspection and is not passed to scoring.

EXACT NOTEBOOK CODE

```
1 | train_poc[
2 |     [
3 |         "Headline",
4 |         "Category",
5 |         "true_risks",
6 |         "normalized_text"
7 |     ]
8 | ].head()
```

CELL 9**Code cell 9**

Execution count: 9

WHAT THIS CELL IMPLEMENTS

Re-declares the six-risk taxonomy locally so Stage A is self-contained for reporting and evaluation labels.

EXACT NOTEBOOK CODE

```
1 | RISK_TAXONOMY = {
2 |     "weather_disruption": {
3 |         "name": "Weather Disruption",
4 |         "description": (
5 |             "Disruption to trade, transport, ports, logistics, or infrastructure "
6 |             "caused by severe weather such as storms, flooding, high winds, "
7 |             "cyclones, or similar weather-related hazards."
8 |         )
9 |     },
10 |
11 |     "natural_disaster": {
12 |         "name": "Natural Disaster",
13 |         "description": (
14 |             "Disruption caused by natural disasters such as earthquakes, "
15 |             "tsunamis, volcanic activity, landslides, or other geological hazards."
```

```

16 |     )
17 | },
18 |
19 | "port_operational_disruption": {
20 |     "name": "Port Operational Disruption",
21 |     "description": (
22 |         "Disruption to normal port or cargo operations caused by congestion, "
23 |         "capacity limitations, operational delays, cargo disruption, "
24 |         "or reduced terminal efficiency."
25 |     )
26 | },
27 |
28 | "port_closure": {
29 |     "name": "Port Closure",
30 |     "description": (
31 |         "Full or partial closure, suspension, or shutdown of a port, terminal, "
32 |         "pier, berth, or related maritime facility."
33 |     )
34 | },
35 |
36 | "labor_strike_disruption": {
37 |     "name": "Labor / Strike Disruption",
38 |     "description": (
39 |         "Disruption caused by worker strikes, industrial action, labor disputes, "
40 |         "walkouts, or related workforce actions affecting transport or logistics."
41 |     )
42 | },
43 |
44 | "maritime_security_navigation_disruption": {
45 |     "name": "Maritime Security / Navigation Disruption",
46 |     "description": (
47 |         "Disruption or increased operational risk to maritime transport caused "
48 |         "by maritime advisories, piracy, security threats, navigation restrictions, "
49 |         "or waterway closures and disruptions."
50 |     )
51 | }
52 | }

```

CELL 10**Code cell 10**

Execution count: 10

WHAT THIS CELL IMPLEMENTS

Prints each risk ID and display name, verifying taxonomy order before scoring and binarization.

EXACT NOTEBOOK CODE

```
1 | for risk_id, info in RISK_TAXONOMY.items():
2 |     print(risk_id, "->", info["name"])
```

CELL 11**Code cell 11**

Execution count: 11

WHAT THIS CELL IMPLEMENTS

Counts labels per event by applying `len` to `true\_risks`, then reports how many records are single-label, double-label, and so on.

EXACT NOTEBOOK CODE

```
1 | # checking how many records are multi-label
2 |
3 | train_poc["num_true_risks"] = train_poc["true_risks"].apply(len)
4 |
5 | train_poc["num_true_risks"].value_counts().sort_index()
```

CELL 14**Code cell 14**

Execution count: 12

WHAT THIS CELL IMPLEMENTS

Displays the training DataFrame head. This duplicates an earlier inspection and performs no transformation.

EXACT NOTEBOOK CODE

```
1 | train_poc.head()
```

CELL 15**Code cell 15**

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

This is an empty code cell. Executing it has no effect on state or output.

EXACT NOTEBOOK CODE

```
[empty code cell]
```

CELL 16**Code cell 16**

Execution count: 13

WHAT THIS CELL IMPLEMENTS

Prints value counts, including nulls, for maritime and port-information flags to evaluate whether those metadata fields are useful.

EXACT NOTEBOOK CODE

```
1 | print(train_poc["maritime_label"].value_counts(dropna=False))
2 | print()
3 | print(train_poc["contains_port_info"].value_counts(dropna=False))
```

CELL 17**Code cell 17**

Execution count: 14

WHAT THIS CELL IMPLEMENTS

Drops `contains\_port\_info` from train and test while tolerating a missing column. Stage A subsequently derives port context from text and keeps `maritime\_label` for maritime support.

EXACT NOTEBOOK CODE

```
1 | # dropping contains_port_info
2 | train_poc = train_poc.drop(columns=["contains_port_info"], errors="ignore")
3 | test_poc = test_poc.drop(columns=["contains_port_info"], errors="ignore")
```

WHAT THIS CELL IMPLEMENTS

Defines per-risk rule configurations: strong phrases, lower-weight keywords, optional context type, and a decision threshold. Strong phrases represent direct evidence; generic maritime keywords are deliberately omitted for the security/navigation class to keep it conservative.

EXACT NOTEBOOK CODE

```
1 | STAGE_A_RULES = {
2 |
3 |     "weather_disruption": {
4 |         "strong_phrases": [
5 |             "severe weather",
6 |             "adverse weather",
7 |             "inclement weather",
8 |             "weather advisory",
9 |             "weather advisories",
10 |            "weather warning",
11 |            "weather warnings",
12 |            "tropical cyclone",
13 |            "tropical storm",
14 |            "storm surge",
15 |            "thunderstorm warning",
16 |            "severe thunderstorm",
17 |            "high winds",
18 |            "strong wind",
19 |            "strong winds",
20 |            "severe wind",
21 |            "severe winds",
22 |            "heavy rainfall",
23 |            "heavy rain",
24 |            "torrential rain",
25 |            "flash flooding",
26 |            "flooding",
27 |            "typhoon",
28 |            "hurricane",
29 |            "cyclone",
30 |            "tornado"
31 |        ],
```

```

32 |
33 |     "keywords": [
34 |         "storm",
35 |         "flood",
36 |         "rainfall",
37 |         "weather",
38 |         "winds",
39 |         "hail",
40 |         "blizzard"
41 |     ],
42 |
43 |     "context": None,
44 |     "threshold": 3
45 | },
46 |
47 | "natural_disaster": {
48 |     "strong_phrases": [
49 |         # These single terms are sufficiently specific to count
50 |         # as strong evidence in this maritime/logistics dataset.
51 |         "earthquake",
52 |         "tsunami",
53 |         "volcanic eruption",
54 |         "volcano",
55 |         "landslide"
56 |     ],
57 |
58 |     "keywords": [
59 |         "magnitude",
60 |         "seismic",
61 |         "volcanic"
62 |     ],
63 |
64 |     "context": None,
65 |     "threshold": 3
66 | },
67 |
68 | "port_operational_disruption": {
69 |     "strong_phrases": [
70 |         # Direct congestion and disruption descriptions

```

```

71 |         "port congestion",
72 |         "terminal congestion",
73 |         "berth congestion",
74 |         "berthing congestion",
75 |         "cargo congestion",
76 |         "port disruption",
77 |         "cargo disruption",
78 |         "operational disruption",
79 |         "operations disrupted",
80 |         "congestion",
81 |         "backlog",
82 |
83 |         # Backlogs, delays, and waiting-time descriptions
84 |         "container backlog",
85 |         "cargo backlog",
86 |         "vessel backlog",
87 |         "berthing delay",
88 |         "berthing delays",
89 |         "berth delay",
90 |         "berth delays",
91 |         "port delay",
92 |         "port delays",
93 |         "terminal delay",
94 |         "terminal delays",
95 |         "vessel delay",
96 |         "vessel delays",
97 |         "shipping delay",
98 |         "shipping delays",
99 |         "cargo delay",
100 |         "cargo delays",
101 |         "operational delays",
102 |         "waiting time",
103 |         "waiting times",
104 |         "vessels waiting",
105 |         "ships waiting",
106 |         "waiting for a berth",
107 |         "waiting for berth",
108 |         "berthing time",
109 |         "dwelling time",

```

```

110 |
111 |     # Capacity and terminal-efficiency descriptions
112 |     "capacity shortage",
113 |     "capacity shortages",
114 |     "capacity constraint",
115 |     "capacity constraints",
116 |     "yard remains full",
117 |     "yard is full",
118 |     "high yard density",
119 |     "reduced capacity",
120 |     "low productivity",
121 |     "port operations normal",
122 |     "waterside operations",
123 |     "gate closures",
124 |
125 |     # Common wording found in the GMRID records
126 |     "impact port operations",
127 |     "impacts port operations",
128 |     "affect port operations",
129 |     "affects port operations",
130 |     "disrupt port operations",
131 |     "disrupts port operations",
132 |     "disrupt operations at",
133 |     "disrupts operations at",
134 |     "impact operations at",
135 |     "impacts operations at",
136 |     "affect operations at",
137 |     "affects operations at",
138 |     "shipping cancellations",
139 |     "omit port",
140 |     "skips port"
141 | ],
142 |
143 |     "keywords": [
144 |         "delay",
145 |         "delays",
146 |         "delayed",
147 |         "waiting",
148 |         "capacity",

```

```

149 |         "shortage",
150 |         "disruption",
151 |         "disrupted"
152 |     ],
153 |
154 |     "context": "port",
155 |     "threshold": 3
156 | },
157 |
158 | "port_closure": {
159 |     "strong_phrases": [
160 |         "port closure",
161 |         "closure of the port",
162 |         "closure of port",
163 |         "port closed",
164 |         "port closes",
165 |         "port to close",
166 |         "port will close",
167 |         "close the port",
168 |         "close port",
169 |         "terminal closure",
170 |         "terminal closed",
171 |         "terminal closes",
172 |         "terminal to close",
173 |         "port operations suspended",
174 |         "port suspends operations",
175 |         "port suspends all operations",
176 |         "suspend operations at the port",
177 |         "operations suspended at port",
178 |         "operations suspended at the port",
179 |         "terminal operations suspended",
180 |         "terminals suspend operations",
181 |         "port shutdown",
182 |         "port shut down",
183 |         "shut down port",
184 |         # "port reopens",
185 |         # "port reopened",
186 |         # "port has reopened",
187 |         "port bans"

```

```

188 |     ],
189 |
190 |     "keywords": [
191 |         "closure",
192 |         "closed",
193 |         "closes",
194 |         "suspended",
195 |         "suspend",
196 |         "shutdown",
197 |         "standstill",
198 |         "reopens",
199 |         "reopened"
200 |     ],
201 |
202 |     "context": "port",
203 |     "threshold": 4
204 | },
205 |
206 | "labor_strike_disruption": {
207 |     "strong_phrases": [
208 |         # "strike" is direct enough to be strong evidence.
209 |         "strike",
210 |         "strikes",
211 |         "industrial action",
212 |         "industrial actions",
213 |         "labor dispute",
214 |         "labour dispute",
215 |         "work stoppage",
216 |         "work stoppages",
217 |         "walkout",
218 |         "walkouts",
219 |         "workers protest",
220 |         "worker protest",
221 |         "dockworkers protest",
222 |         "longshore workers"
223 |     ],
224 |
225 |     "keywords": [
226 |         "striking",

```

```

227 |         "dockworker",
228 |         "dockworkers",
229 |         "longshoremen",
230 |         "union",
231 |         "unions",
232 |         "labor",
233 |         "labour",
234 |         "workers",
235 |         "truckers",
236 |         "transporters"
237 |     ],
238 |
239 |     "context": None,
240 |     "threshold": 3
241 | },
242 |
243 | "maritime_security_navigation_disruption": {
244 |     "strong_phrases": [
245 |         # Direct maritime security or navigation evidence
246 |         "maritime advisory",
247 |         "maritime security",
248 |         "navigation warning",
249 |         "navigation warnings",
250 |         "navigation restriction",
251 |         "navigation restrictions",
252 |         "waterway closure",
253 |         "waterway disruption",
254 |
255 |         # Waterway and vessel-traffic restrictions
256 |         "ship channel closes",
257 |         "ship channel closed",
258 |         "ship channel reopens",
259 |         "ship channel reopened",
260 |         "shipping channel closes",
261 |         "shipping channel closed",
262 |         "shipping channel reopens",
263 |         "vessel traffic suspended",
264 |         "vessel traffic halted",
265 |         "vessel traffic reopened",

```

```

266 |         "vessel traffic resumes",
267 |         "river vessel traffic",
268 |         "restricted berthing",
269 |
270 |         # Piracy and vessel-security incidents
271 |         "piracy",
272 |         "pirates",
273 |         "pirate attack",
274 |         "armed robbers",
275 |         "robbers board",
276 |         "robbers boarded",
277 |         "robbery aboard",
278 |         "robbery reported aboard",
279 |         "ship attacked",
280 |         "vessel attacked",
281 |         "tanker attacked",
282 |         "carrier attacked",
283 |
284 |         # Common maritime schedule-advisory wording
285 |         "blank sailing",
286 |         "blanks week"
287 |     ],
288 |
289 |     # Generic words such as ship, vessel, traffic, or channel occur in
290 |     # many non-security records. Keep this Stage A class conservative
291 |     # and let Stage B handle cases without a direct phrase.
292 |     "keywords": [],
293 |
294 |     "context": "maritime",
295 |     "threshold": 3
296 | }
297 | }

```

CELL 19**Code cell 19**

Execution count: 16

WHAT THIS CELL IMPLEMENTS

Prints the number of strong phrases and keywords plus context and threshold for every risk, providing a compact rule inventory.

EXACT NOTEBOOK CODE

```
1 | # for risk_id, rules in STAGE_A_RULES.items():
2 |
3 | #     print(risk_id)
4 | #     print("Strong phrases:", len(rules["strong_phrases"]))
5 | #     print("Keywords:", len(rules["keywords"]))
6 | #     print("Context:", rules["context"])
7 | #     print("-" * 50)
8 |
9 | for risk_id, rules in STAGE_A_RULES.items():
10 |
11 |     print(risk_id)
12 |     print("Strong phrases:", len(rules["strong_phrases"]))
13 |     print("Keywords:", len(rules["keywords"]))
14 |     print("Context:", rules["context"])
15 |     print("Threshold:", rules["threshold"])
16 |     print("-" * 50)
```

CELL 21**Code cell 21**

Execution count: 17

WHAT THIS CELL IMPLEMENTS

Implements the scoring engine. It caches boundary-aware regex patterns, prevents substring matches inside larger words, de-duplicates nested terms, avoids double-counting keywords already inside matched phrases, awards 3 points per strong phrase and 1 per independent keyword, and adds at most 1 context point only when textual evidence already exists.

EXACT NOTEBOOK CODE

```
1 | # def calculate_rule_score(text, rule_config, row=None):
2 |
3 | #     text = str(text).lower()
4 |
```

```

5 | #     score = 0
6 | #     matched_phrases = []
7 | #     matched_keywords = []
8 | #     context_bonus = 0
9 |
10 | #     # Strong phrase matches
11 | #     for phrase in rule_config["strong_phrases"]:
12 | #         if phrase in text:
13 | #             score += 3
14 | #             matched_phrases.append(phrase)
15 |
16 | #     # Keyword matches
17 | #     for keyword in rule_config["keywords"]:
18 | #         if keyword in text:
19 | #             score += 1
20 | #             matched_keywords.append(keyword)
21 |
22 | #     # Metadata bonus only for maritime context
23 | #     if score > 0 and row is not None:
24 |
25 | #         context = rule_config.get("context")
26 |
27 | #         if context == "maritime":
28 | #             if row.get("maritime_label", False):
29 | #                 score += 1
30 | #                 context_bonus = 1
31 |
32 | #     return {
33 | #         "score": score,
34 | #         "matched_phrases": matched_phrases,
35 | #         "matched_keywords": matched_keywords,
36 | #         "context_bonus": context_bonus
37 | #     }
38 |
39 | #-----
40 | # Cache compiled patterns because the same terms are applied to every row.
41 | TERM_PATTERN_CACHE = {}
42 |
43 |

```

```

44 | def get_term_pattern(term):
45 |
46 |     if term not in TERM_PATTERN_CACHE:
47 |
48 |         # Lookarounds prevent a term such as "union" from matching inside
49 |         # a larger unrelated word. Spaces are allowed to match any
50 |         # normalized whitespace between phrase words.
51 |         escaped_term = re.escape(
52 |             term.lower()
53 |         ).replace(
54 |             r"\ ",
55 |             r"\s+"
56 |         )
57 |
58 |         TERM_PATTERN_CACHE[term] = re.compile(
59 |             r"(?<![a-z0-9])"
60 |             + escaped_term
61 |             + r"(?![a-z0-9])"
62 |         )
63 |
64 |     return TERM_PATTERN_CACHE[term]
65 |
66 |
67 | def term_is_present(text, term):
68 |
69 |     pattern = get_term_pattern(
70 |         term
71 |     )
72 |
73 |     return pattern.search(text) is not None
74 |
75 |
76 | def find_distinct_matches(text, terms):
77 |
78 |     matched_terms = []
79 |
80 |     # Check longer phrases first. This prevents "cyclone" and
81 |     # "tropical cyclone" from being counted as two separate pieces
82 |     # of evidence when they refer to the same text span.

```

```

83 |     ordered_terms = sorted(
84 |         dict.fromkeys(terms),
85 |         key=lambda term: (
86 |             -len(term),
87 |             term
88 |         )
89 |     )
90 |
91 |     for term in ordered_terms:
92 |
93 |         if not term_is_present(text, term):
94 |             continue
95 |
96 |         contained_in_existing_match = any(
97 |             term_is_present(existing_match, term)
98 |             for existing_match in matched_terms
99 |         )
100 |
101 |         if not contained_in_existing_match:
102 |             matched_terms.append(
103 |                 term
104 |             )
105 |
106 |     return matched_terms
107 |
108 |
109 | PORT_CONTEXT_TERMS = [
110 |     "port",
111 |     "terminal",
112 |     "pier",
113 |     "berth",
114 |     "harbor",
115 |     "harbour",
116 |     "dock"
117 | ]
118 |
119 |
120 | MARITIME_CONTEXT_TERMS = [
121 |     "ship",

```

```

122 |     "shipping",
123 |     "vessel",
124 |     "maritime",
125 |     "marine",
126 |     "waterway",
127 |     "channel",
128 |     "strait",
129 |     "anchorage",
130 |     "tanker",
131 |     "carrier",
132 |     "barge",
133 |     "port",
134 |     "harbor",
135 |     "harbour"
136 | ]
137 |
138 |
139 | def has_any_term(text, terms):
140 |
141 |     return any(
142 |         term_is_present(text, term)
143 |         for term in terms
144 |     )
145 |
146 |
147 | def get_context_bonus(text, context, row):
148 |
149 |     if context == "port":
150 |
151 |         return int(
152 |             has_any_term(
153 |                 text,
154 |                 PORT_CONTEXT_TERMS
155 |             )
156 |         )
157 |
158 |     if context == "maritime":
159 |
160 |         has_maritime_metadata = to_bool(

```

```

161 |         row.get(
162 |             "maritime_label",
163 |             False
164 |         )
165 |     )
166 |
167 |     has_maritime_text = has_any_term(
168 |         text,
169 |         MARITIME_CONTEXT_TERMS
170 |     )
171 |
172 |     return int(
173 |         has_maritime_metadata
174 |         or has_maritime_text
175 |     )
176 |
177 |     return 0
178 |
179 |
180 | def calculate_rule_score(text, rule_config, row=None):
181 |
182 |     if pd.isna(text):
183 |         text = ""
184 |     else:
185 |         text = str(text).lower()
186 |
187 |     matched_phrases = find_distinct_matches(
188 |         text,
189 |         rule_config["strong_phrases"]
190 |     )
191 |
192 |     possible_keywords = find_distinct_matches(
193 |         text,
194 |         rule_config["keywords"]
195 |     )
196 |
197 |     matched_keywords = []
198 |
199 |     for keyword in possible_keywords:

```

```

200 |
201 |     # Do not count a keyword again when it is already part of a
202 |     # matched strong phrase. For example, "weather" should not add
203 |     # another point after "weather warning" has matched.
204 |     already_in_phrase = any(
205 |         term_is_present(phrase, keyword)
206 |         for phrase in matched_phrases
207 |     )
208 |
209 |     if not already_in_phrase:
210 |         matched_keywords.append(
211 |             keyword
212 |         )
213 |
214 |     score = (
215 |         3 * len(matched_phrases)
216 |         + len(matched_keywords)
217 |     )
218 |
219 |     context_bonus = 0
220 |
221 |     # Metadata or contextual wording can support textual evidence,
222 |     # but it cannot create a prediction by itself.
223 |     if score > 0 and row is not None:
224 |
225 |         context_bonus = get_context_bonus(
226 |             text=text,
227 |             context=rule_config.get("context"),
228 |             row=row
229 |         )
230 |
231 |         score += context_bonus
232 |
233 |     return {
234 |         "score": score,
235 |         "matched_phrases": matched_phrases,
236 |         "matched_keywords": matched_keywords,
237 |         "context_bonus": context_bonus
238 |     }

```

WHAT THIS CELL IMPLEMENTS

Defines `get\_stage\_a\_scores`, which scores the row's normalized text independently against every risk configuration and returns the complete evidence dictionary.

EXACT NOTEBOOK CODE

```
1 | # def get_stage_a_scores(row):
2 |
3 | #     text = row["normalized_text"]
4 |
5 | #     results = {}
6 |
7 | #     for risk_id, rule_config in STAGE_A_RULES.items():
8 |
9 | #         results[risk_id] = calculate_rule_score(
10 | #             text=text,
11 | #             rule_config=rule_config,
12 | #             row=row
13 | #         )
14 |
15 | #     return results
16 |
17 | def get_stage_a_scores(row):
18 |
19 |     text = row.get(
20 |         "normalized_text",
21 |         ""
22 |     )
23 |
24 |     results = {}
25 |
26 |     for risk_id, rule_config in STAGE_A_RULES.items():
27 |
28 |         results[risk_id] = calculate_rule_score(
29 |             text=text,
30 |             rule_config=rule_config,
31 |             row=row
32 |         )
```

```
33 |  
34 |     return results
```

CELL 23

Code cell 23

Execution count: 19

WHAT THIS CELL IMPLEMENTS

Selects the first training event, prints its headline and true labels, and displays all Stage A scores as an explainability example.

EXACT NOTEBOOK CODE

```
1 | example_row = train_poc.iloc[0]  
2 |  
3 | print("Headline:")  
4 | print(example_row["Headline"])  
5 |  
6 | print("\nTrue risks:")  
7 | print(example_row["true_risks"])  
8 |  
9 | print("\nStage A scores:")  
10 | get_stage_a_scores(example_row)
```

CELL 24

Code cell 24

Execution count: 20

WHAT THIS CELL IMPLEMENTS

Defines classification and runs it over both splits. Any risk meeting its own threshold is predicted; evidence is retained only for predicted labels; rows with no Stage A prediction receive `route\_to\_stage\_b=True`; results are unpacked into prediction, evidence, and routing columns.

EXACT NOTEBOOK CODE

```
1 | def classify_stage_a(row):  
2 |  
3 |     risk_scores = get_stage_a_scores(  
4 |         row  
5 |     )  
6 |  
7 |     predictions = []
```

```

8 |
9 |     for risk_id, evidence in risk_scores.items():
10 |
11 |         threshold = STAGE_A_RULES[
12 |             risk_id
13 |         ].get(
14 |             "threshold",
15 |             3
16 |         )
17 |
18 |         if evidence["score"] >= threshold:
19 |             predictions.append(
20 |                 risk_id
21 |             )
22 |
23 |     prediction_evidence = {
24 |         risk_id: risk_scores[risk_id]
25 |         for risk_id in predictions
26 |     }
27 |
28 |     return {
29 |         "predictions": predictions,
30 |         "evidence": prediction_evidence,
31 |         "route_to_stage_b": len(predictions) == 0
32 |     }
33 |
34 |
35 | train_stage_a_results = train_poc.apply(
36 |     classify_stage_a,
37 |     axis=1
38 | )
39 |
40 | test_stage_a_results = test_poc.apply(
41 |     classify_stage_a,
42 |     axis=1
43 | )
44 |
45 |
46 | train_poc["stage_a_predictions"] = train_stage_a_results.apply(

```

```

47 |     lambda result: result["predictions"]
48 | )
49 |
50 | train_poc["stage_a_evidence"] = train_stage_a_results.apply(
51 |     lambda result: result["evidence"]
52 | )
53 |
54 | train_poc["route_to_stage_b"] = train_stage_a_results.apply(
55 |     lambda result: result["route_to_stage_b"]
56 | )
57 |
58 |
59 | test_poc["stage_a_predictions"] = test_stage_a_results.apply(
60 |     lambda result: result["predictions"]
61 | )
62 |
63 | test_poc["stage_a_evidence"] = test_stage_a_results.apply(
64 |     lambda result: result["evidence"]
65 | )
66 |
67 | test_poc["route_to_stage_b"] = test_stage_a_results.apply(
68 |     lambda result: result["route_to_stage_b"]
69 | )

```

CELL 25

Code cell 25

Execution count: 21

WHAT THIS CELL IMPLEMENTS

Defines multi-label evaluation. It binarizes the six labels, prints per-class and aggregate classification metrics, computes micro/macro precision-recall-F1, exact set match, Stage A coverage, and the count routed onward, then returns the aggregate values.

EXACT NOTEBOOK CODE

```

1 | from sklearn.metrics import classification_report
2 | from sklearn.metrics import precision_recall_fscore_support
3 | from sklearn.preprocessing import MultiLabelBinarizer
4 |
5 |
6 | RISK_IDS = list(

```

```

7 |     RISK_TAXONOMY.keys()
8 | )
9 |
10 |
11 | def evaluate_stage_a(dataframe, split_name):
12 |
13 |     label_binarizer = MultiLabelBinarizer(
14 |         classes=RISK_IDS
15 |     )
16 |
17 |     true_matrix = label_binarizer.fit_transform(
18 |         dataframe["true_risks"]
19 |     )
20 |
21 |     predicted_matrix = label_binarizer.transform(
22 |         dataframe["stage_a_predictions"]
23 |     )
24 |
25 |     print(
26 |         f"{split_name} classification report"
27 |     )
28 |
29 |     print(
30 |         classification_report(
31 |             true_matrix,
32 |             predicted_matrix,
33 |             target_names=[
34 |                 RISK_TAXONOMY[risk_id]["name"]
35 |                 for risk_id in RISK_IDS
36 |             ],
37 |             zero_division=0
38 |         )
39 |     )
40 |
41 |     micro_precision, micro_recall, micro_f1, _ = (
42 |         precision_recall_fscore_support(
43 |             true_matrix,
44 |             predicted_matrix,
45 |             average="micro",

```

```

46 |         zero_division=0
47 |     )
48 | )
49 |
50 | macro_precision, macro_recall, macro_f1, _ = (
51 |     precision_recall_fscore_support(
52 |         true_matrix,
53 |         predicted_matrix,
54 |         average="macro",
55 |         zero_division=0
56 |     )
57 | )
58 |
59 | covered_mask = dataframe[
60 |     "stage_a_predictions"
61 | ].apply(
62 |     lambda predictions: len(predictions) > 0
63 | )
64 |
65 | exact_match = np.mean(
66 |     [
67 |         set(true_risks) == set(predicted_risks)
68 |         for true_risks, predicted_risks in zip(
69 |             dataframe["true_risks"],
70 |             dataframe["stage_a_predictions"]
71 |         )
72 |     ]
73 | )
74 |
75 | coverage = covered_mask.mean()
76 |
77 | print(
78 |     "Micro precision:",
79 |     round(micro_precision, 4)
80 | )
81 |
82 | print(
83 |     "Micro recall:",
84 |     round(micro_recall, 4)

```

```

85 |     )
86 |
87 |     print(
88 |         "Micro F1:",
89 |         round(micro_f1, 4)
90 |     )
91 |
92 |     print(
93 |         "Macro precision:",
94 |         round(macro_precision, 4)
95 |     )
96 |
97 |     print(
98 |         "Macro recall:",
99 |         round(macro_recall, 4)
100 |    )
101 |
102 |    print(
103 |        "Macro F1:",
104 |        round(macro_f1, 4)
105 |    )
106 |
107 |    print(
108 |        "Exact multi-label match:",
109 |        round(exact_match, 4)
110 |    )
111 |
112 |    print(
113 |        "Stage A coverage:",
114 |        round(coverage, 4)
115 |    )
116 |
117 |    print(
118 |        "Rows routed to Stage B:",
119 |        int((~covered_mask).sum())
120 |    )
121 |
122 |    return {
123 |        "micro_precision": micro_precision,

```

```

124 |         "micro_recall": micro_recall,
125 |         "micro_f1": micro_f1,
126 |         "macro_precision": macro_precision,
127 |         "macro_recall": macro_recall,
128 |         "macro_f1": macro_f1,
129 |         "exact_match": exact_match,
130 |         "coverage": coverage
131 |     }

```

CELL 26

Code cell 26

Execution count: 22

WHAT THIS CELL IMPLEMENTS

Evaluates Stage A on training data for rule-development feedback and stores the returned aggregate metrics.

EXACT NOTEBOOK CODE

```

1 | # Use training results while changing the rule lists.
2 | train_stage_a_metrics = evaluate_stage_a(
3 |     train_poc,
4 |     "Training"
5 | )

```

CELL 27

Code cell 27

Execution count: 23

WHAT THIS CELL IMPLEMENTS

Defines a targeted error-analysis helper. For a selected risk it builds truth and prediction masks, selects false positives or false negatives, rejects invalid error types, and returns diagnostic columns including matched evidence.

EXACT NOTEBOOK CODE

```

1 | def get_label_error_rows(
2 |     dataframe,
3 |     risk_id,
4 |     error_type
5 | ):
6 |

```

```

7 |     true_mask = dataframe[
8 |         "true_risks"
9 |     ].apply(
10 |         lambda risks: risk_id in risks
11 |     )
12 |
13 |     predicted_mask = dataframe[
14 |         "stage_a_predictions"
15 |     ].apply(
16 |         lambda risks: risk_id in risks
17 |     )
18 |
19 |     if error_type == "false_positive":
20 |
21 |         error_mask = (
22 |             predicted_mask
23 |             & ~true_mask
24 |         )
25 |
26 |     elif error_type == "false_negative":
27 |
28 |         error_mask = (
29 |             true_mask
30 |             & ~predicted_mask
31 |         )
32 |
33 |     else:
34 |         raise ValueError(
35 |             "error_type must be "
36 |             "'false_positive' or 'false_negative'"
37 |         )
38 |
39 |     columns_to_show = [
40 |         "id",
41 |         "Headline",
42 |         "Category",
43 |         "Summarized_label",
44 |         "true_risks",
45 |         "stage_a_predictions",

```

```
46 |         "stage_a_evidence"
47 |     ]
48 |
49 |     return dataframe.loc[
50 |         error_mask,
51 |         columns_to_show
52 |     ].copy()
```

CELL 28

Code cell 28

Execution count: 24

WHAT THIS CELL IMPLEMENTS

Extracts and counts training false positives for `port\_closure`, then displays the first 20 cases for rule refinement.

EXACT NOTEBOOK CODE

```
1 | port_closure_false_positives = get_label_error_rows(
2 |     train_poc,
3 |     risk_id="port_closure",
4 |     error_type="false_positive"
5 | )
6 |
7 | print(
8 |     "Port closure false positives:",
9 |     len(port_closure_false_positives)
10 | )
11 |
12 | port_closure_false_positives.head(20)
```

CELL 29

Code cell 29

Execution count: 25

WHAT THIS CELL IMPLEMENTS

Extracts and counts training false negatives for `port\_closure`, then displays the first 20 missed cases.

EXACT NOTEBOOK CODE

```
1 | port_closure_false_negatives = get_label_error_rows(  
2 |     train_poc,  
3 |     risk_id="port_closure",  
4 |     error_type="false_negative"  
5 | )  
6 |  
7 | print(  
8 |     "Port closure false negatives:",  
9 |     len(port_closure_false_negatives)  
10 | )  
11 |  
12 | port_closure_false_negatives.head(20)
```

CELL 30

Code cell 30

Execution count: 26

WHAT THIS CELL IMPLEMENTS

Evaluates the finalized Stage A rules on the held-out test split. The comments correctly warn against repeatedly tuning rules on test errors.

EXACT NOTEBOOK CODE

```
1 | # Run the test evaluation only after the Stage A rules are finalized.  
2 | # Do not repeatedly tune the rules using test-set errors.  
3 | test_stage_a_metrics = evaluate_stage_a(  
4 |     test_poc,  
5 |     "Test"  
6 | )
```

CELL 31

Code cell 31

Execution count: 27

WHAT THIS CELL IMPLEMENTS

Builds a two-row train-versus-test metric table and rounds it to four decimals for an overfitting/generalization comparison.

EXACT NOTEBOOK CODE

```
1 | stage_a_metric_comparison = pd.DataFrame(  
2 |     [  
3 |         train_stage_a_metrics,  
4 |         test_stage_a_metrics  
5 |     ],  
6 |     index=[  
7 |         "Training",  
8 |         "Test"  
9 |     ]  
10 | )  
11 |  
12 | stage_a_metric_comparison.round(4)
```

CELL 32

Code cell 32

Execution count: 28

WHAT THIS CELL IMPLEMENTS

Creates the output directory, serializes each prediction list as JSON, and writes compact ID-to-prediction CSVs for both splits. Evidence and the explicit `route\_to\_stage\_b` flag are not exported; downstream routing is reconstructed from whether the prediction list is empty.

EXACT NOTEBOOK CODE

```
1 | import json  
2 | from pathlib import Path  
3 |  
4 |  
5 | OUTPUT_DIR = Path("../outputs")  
6 |  
7 | OUTPUT_DIR.mkdir(  
8 |     parents=True,  
9 |     exist_ok=True  
10 | )  
11 |  
12 |  
13 | stage_a_train_output = pd.DataFrame({  
14 |     "id": train_poc["id"],  
15 |     "stage_a_predictions": train_poc[  
16 |         "stage_a_predictions"
```

```

17 |     ].apply(
18 |         lambda value: json.dumps(
19 |             value,
20 |             ensure_ascii=False
21 |         )
22 |     )
23 | })
24 |
25 |
26 | stage_a_test_output = pd.DataFrame({
27 |     "id": test_poc["id"],
28 |     "stage_a_predictions": test_poc[
29 |         "stage_a_predictions"
30 |     ].apply(
31 |         lambda value: json.dumps(
32 |             value,
33 |             ensure_ascii=False
34 |         )
35 |     )
36 | })
37 |
38 |
39 | stage_a_train_output.to_csv(
40 |     OUTPUT_DIR / "stage_a_train_predictions.csv",
41 |     index=False
42 | )
43 |
44 | stage_a_test_output.to_csv(
45 |     OUTPUT_DIR / "stage_a_test_predictions.csv",
46 |     index=False
47 | )
48 |
49 |
50 | print(
51 |     "Stage A prediction files exported successfully."
52 | )

```

Stage B - Embeddings and Classifiers

Sentence embeddings, one-vs-rest learning, thresholds, and routing

SOURCE notebooks/03_stage_b_embeddings.ipynb

28 CODE CELLS

CELL 1

Code cell 1

Execution count: 14

WHAT THIS CELL IMPLEMENTS

Imports JSON and path utilities, pandas, and SentenceTransformers. These support file interchange and semantic embedding generation.

EXACT NOTEBOOK CODE

```
1 | import json
2 | from pathlib import Path
3 |
4 | import pandas as pd
5 |
6 | from sentence_transformers import SentenceTransformer
```

CELL 2

Code cell 2

Execution count: 15

WHAT THIS CELL IMPLEMENTS

Resolves the project root whether run from the repository or notebook folder, loads POC datasets and Stage A prediction files, and prints record counts for all four inputs.

EXACT NOTEBOOK CODE

```
1 | CURRENT_DIR = Path.cwd()
2 |
3 | if CURRENT_DIR.name == "notebooks":
4 |     PROJECT_ROOT = CURRENT_DIR.parent
5 | else:
6 |     PROJECT_ROOT = CURRENT_DIR
7 |
```

```
8 |  
9 | DATA_DIR = PROJECT_ROOT / "data"  
10 | OUTPUT_DIR = PROJECT_ROOT / "outputs"  
11 |  
12 |  
13 | train_poc = pd.read_csv(  
14 |     DATA_DIR / "train_poc.csv"  
15 | )  
16 |  
17 | test_poc = pd.read_csv(  
18 |     DATA_DIR / "test_poc.csv"  
19 | )  
20 |  
21 |  
22 | stage_a_train = pd.read_csv(  
23 |     OUTPUT_DIR / "stage_a_train_predictions.csv"  
24 | )  
25 |  
26 | stage_a_test = pd.read_csv(  
27 |     OUTPUT_DIR / "stage_a_test_predictions.csv"  
28 | )  
29 |  
30 |  
31 | print("Train records:", len(train_poc))  
32 | print("Test records:", len(test_poc))  
33 | print("Stage A train predictions:", len(stage_a_train))  
34 | print("Stage A test predictions:", len(stage_a_test))
```

CELL 3

Code cell 3

Execution count: 17

WHAT THIS CELL IMPLEMENTS

Deserializes Stage A prediction lists from JSON strings back into Python lists.

EXACT NOTEBOOK CODE

```
1 | # CSV stores prediction lists as text.
2 | stage_a_train["stage_a_predictions"] = (
3 |     stage_a_train["stage_a_predictions"].apply(
4 |         json.loads
5 |     )
6 | )
7 |
8 | stage_a_test["stage_a_predictions"] = (
9 |     stage_a_test["stage_a_predictions"].apply(
10 |         json.loads
11 |     )
12 | )
```

CELL 4

Code cell 4

Execution count: 18

WHAT THIS CELL IMPLEMENTS

Checks that every POC ID appears in the corresponding Stage A file and raises a clear error if coverage is incomplete. It checks inclusion, while later one-to-one merges provide stronger uniqueness validation.

EXACT NOTEBOOK CODE

```
1 | # validation cell
2 | # Confirm that every POC record has a Stage A result.
3 | if not train_poc["id"].isin(
4 |     stage_a_train["id"]
5 | ).all():
6 |     raise ValueError(
7 |         "Some training records do not have Stage A predictions."
8 |     )
9 |
10 |
11 | if not test_poc["id"].isin(
12 |     stage_a_test["id"]
13 | ).all():
14 |     raise ValueError(
15 |         "Some test records do not have Stage A predictions."
16 |     )
17 |
18 |
19 | print(
20 |     "Stage A prediction IDs match the POC datasets."
21 | )
```

CELL 5

Code cell 5

Execution count: 19

WHAT THIS CELL IMPLEMENTS

Re-declares the six risk definitions so embedding classification, metrics, and exported probabilities share the same ordered label space.

EXACT NOTEBOOK CODE

```
1 | # Define the risk descriptions
2 | RISK_TAXONOMY = {
3 |     "weather_disruption": {
4 |         "name": "Weather Disruption",
5 |         "description": (
6 |             "Disruption to trade, transport, ports, logistics, or infrastructure "
7 |             "caused by severe weather such as storms, flooding, high winds, "
8 |             "cyclones, or similar weather-related hazards."
```

```

9 |     )
10 | },
11 |
12 |     "natural_disaster": {
13 |         "name": "Natural Disaster",
14 |         "description": (
15 |             "Disruption caused by natural disasters such as earthquakes, "
16 |             "tsunamis, volcanic activity, landslides, or other geological hazards."
17 |         )
18 |     },
19 |
20 |     "port_operational_disruption": {
21 |         "name": "Port Operational Disruption",
22 |         "description": (
23 |             "Disruption to normal port or cargo operations caused by congestion, "
24 |             "capacity limitations, operational delays, cargo disruption, "
25 |             "or reduced terminal efficiency."
26 |         )
27 |     },
28 |
29 |     "port_closure": {
30 |         "name": "Port Closure",
31 |         "description": (
32 |             "Full or partial closure, suspension, or shutdown of a port, terminal, "
33 |             "pier, berth, or related maritime facility."
34 |         )
35 |     },
36 |
37 |     "labor_strike_disruption": {
38 |         "name": "Labor / Strike Disruption",
39 |         "description": (
40 |             "Disruption caused by worker strikes, industrial action, labor disputes, "
41 |             "walkouts, or related workforce actions affecting transport or logistics."
42 |         )
43 |     },
44 |
45 |     "maritime_security_navigation_disruption": {
46 |         "name": "Maritime Security / Navigation Disruption",
47 |         "description": (

```

```

48 |         "Disruption or increased operational risk to maritime transport caused "
49 |         "by maritime advisories, piracy, security threats, navigation restrictions, "
50 |         "or waterway closures and disruptions."
51 |     )
52 | }
53 | }

```

CELL 6

Code cell 6

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Imports modeling utilities, restores true-risk lists from CSV, fixes `RISK\_IDS` order, initializes `MultiLabelBinarizer`, and converts training labels into a 3322-by-6 binary target matrix.

EXACT NOTEBOOK CODE

```

1 | import ast
2 | import numpy as np
3 | import torch
4 |
5 | from sklearn.linear_model import LogisticRegression
6 | from sklearn.metrics import classification_report
7 | from sklearn.metrics import f1_score
8 | from sklearn.metrics import precision_recall_fscore_support
9 | from sklearn.model_selection import KFold
10 | from sklearn.model_selection import cross_val_predict
11 | from sklearn.multiclass import OneVsRestClassifier
12 | from sklearn.preprocessing import MultiLabelBinarizer
13 |
14 |
15 | # Restore the true risk lists loaded from CSV.
16 | def restore_list(value):
17 |     if isinstance(value, list):
18 |         return value
19 |
20 |     if pd.isna(value):
21 |         return []
22 |
23 |     return ast.literal_eval(value)
24 |

```

```

25 |
26 | train_poc["true_risks"] = (
27 |     train_poc["true_risks"].apply(
28 |         restore_list
29 |     )
30 | )
31 |
32 | test_poc["true_risks"] = (
33 |     test_poc["true_risks"].apply(
34 |         restore_list
35 |     )
36 | )
37 |
38 |
39 | RISK_IDS = list(
40 |     RISK_TAXONOMY.keys()
41 | )
42 |
43 |
44 | label_binarizer = MultiLabelBinarizer(
45 |     classes=RISK_IDS
46 | )
47 |
48 | train_target_matrix = label_binarizer.fit_transform(
49 |     train_poc["true_risks"]
50 | )

```

CELL 7

Code cell 7

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Requires a CUDA GPU, selects `all-MiniLM-L6-v2`, and loads the SentenceTransformer on GPU. On CPU-only environments this cell intentionally stops with a RuntimeError rather than falling back.

EXACT NOTEBOOK CODE

```
1 | # model-loading
2 |
3 | # Stop execution if a Colab GPU is not enabled.
4 | if not torch.cuda.is_available():
5 |     raise RuntimeError(
6 |         "GPU is not enabled. Select Runtime > Change runtime type > GPU."
7 |     )
8 |
9 |
10 | DEVICE = "cuda"
11 |
12 | MODEL_NAME = (
13 |     "sentence-transformers/all-MiniLM-L6-v2"
14 | )
15 |
16 |
17 | semantic_model = SentenceTransformer(
18 |     MODEL_NAME,
19 |     device=DEVICE
20 | )
21 |
22 |
23 | print("Device:", DEVICE)
24 | print("Semantic model:", MODEL_NAME)
```

CELL 8

Code cell 8

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Converts input text to clean string lists and encodes train and test events in batches of 64. Embeddings are returned as NumPy arrays and L2-normalized, enabling cosine similarity through a dot product; the model produces 384 dimensions.

EXACT NOTEBOOK CODE

```
1 | # Generate semantic embeddings
2 | train_texts = (
3 |     train_poc["input_text"]
4 |     .fillna("")
```

```

5 |     .astype(str)
6 |     .tolist()
7 | )
8 |
9 | test_texts = (
10 |     test_poc["input_text"]
11 |     .fillna("")
12 |     .astype(str)
13 |     .tolist()
14 | )
15 |
16 |
17 | train_embeddings = semantic_model.encode(
18 |     train_texts,
19 |     batch_size=64,
20 |     show_progress_bar=True,
21 |     convert_to_numpy=True,
22 |     normalize_embeddings=True
23 | )
24 |
25 | test_embeddings = semantic_model.encode(
26 |     test_texts,
27 |     batch_size=64,
28 |     show_progress_bar=True,
29 |     convert_to_numpy=True,
30 |     normalize_embeddings=True
31 | )
32 |
33 |
34 | print(
35 |     "Train embeddings shape:",
36 |     train_embeddings.shape
37 | )
38 |
39 | print(
40 |     "Test embeddings shape:",
41 |     test_embeddings.shape
42 | )

```

CELL 9**Code cell 9**

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Validates embedding row counts and saves IDs, float32 embeddings, and model name in a compressed NPZ. Stage C later uses this artifact for retrieval without recomputing embeddings.

EXACT NOTEBOOK CODE

```
1 | # Save embeddings for Stage C
2 | OUTPUT_DIR.mkdir(
3 |     parents=True,
4 |     exist_ok=True
5 | )
6 |
7 | assert len(train_embeddings) == len(train_poc)
8 | assert len(test_embeddings) == len(test_poc)
9 |
10 | np.savez_compressed(
11 |     OUTPUT_DIR / "stage_b_embeddings.npz",
12 |     train_ids=train_poc["id"].to_numpy(),
13 |     train_embeddings=train_embeddings.astype(np.float32),
14 |     test_ids=test_poc["id"].to_numpy(),
15 |     test_embeddings=test_embeddings.astype(np.float32),
16 |     model_name=np.array(MODEL_NAME)
17 | )
18 |
19 | print("Stage B embeddings saved.")
```

CELL 10**Code cell 10**

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Builds a balanced logistic-regression base learner and wraps it in one-vs-rest multi-label classification. Three-fold shuffled out-of-fold probabilities are generated for threshold tuning, then a final model is fit on all training embeddings and applied to test embeddings.

EXACT NOTEBOOK CODE

```
1 | # Train the multilabel semantic classifier
2 |
```

```

3 | # Logistic regression learns one classifier
4 | # for each target risk.
5 | base_classifier = LogisticRegression(
6 |     max_iter=300,
7 |     solver="liblinear",
8 |     class_weight="balanced",
9 |     random_state=42
10 | )
11 |
12 | stage_b_model = OneVsRestClassifier(
13 |     base_classifier,
14 |     n_jobs=-1
15 | )
16 |
17 |
18 | # Out-of-fold probabilities provide a fairer
19 | # estimate for selecting the thresholds.
20 | cross_validation = KFold(
21 |     n_splits=3,
22 |     shuffle=True,
23 |     random_state=42
24 | )
25 |
26 | train_stage_b_probabilities = cross_val_predict(
27 |     stage_b_model,
28 |     train_embeddings,
29 |     train_target_matrix,
30 |     cv=cross_validation,
31 |     method="predict_proba"
32 | )
33 |
34 |
35 | # Train the final model using all training records.
36 | stage_b_model.fit(
37 |     train_embeddings,
38 |     train_target_matrix
39 | )
40 |
41 | test_stage_b_probabilities = (

```

```

42 |     stage_b_model.predict_proba(
43 |         test_embeddings
44 |     )
45 | )
46 |
47 |
48 | print(
49 |     "Training probabilities shape:",
50 |     train_stage_b_probabilities.shape
51 | )
52 |
53 | print(
54 |     "Test probabilities shape:",
55 |     test_stage_b_probabilities.shape
56 | )

```

CELL 11

Code cell 11

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Searches thresholds from 0.20 through 0.80 in 0.05 steps separately for each risk. The chosen threshold maximizes that label's out-of-fold binary F1, allowing rare and common risks to use different cutoffs.

EXACT NOTEBOOK CODE

```

1 | # Select a threshold for each risk
2 | # Different risks need different probability thresholds.
3 | threshold_options = np.arange(
4 |     0.20,
5 |     0.81,
6 |     0.05
7 | )
8 |
9 | stage_b_thresholds = {}
10 |
11 |
12 | for risk_index, risk_id in enumerate(RISK_IDS):
13 |
14 |     best_threshold = 0.50
15 |     best_f1 = 0.0

```

```

16 |
17 |     for threshold in threshold_options:
18 |
19 |         predicted_labels = (
20 |             train_stage_b_probabilities[
21 |                 :,
22 |                 risk_index
23 |             ] >= threshold
24 |         ).astype(int)
25 |
26 |         threshold_f1 = f1_score(
27 |             train_target_matrix[
28 |                 :,
29 |                 risk_index
30 |             ],
31 |             predicted_labels,
32 |             zero_division=0
33 |         )
34 |
35 |         if threshold_f1 > best_f1:
36 |             best_f1 = threshold_f1
37 |             best_threshold = threshold
38 |
39 |         stage_b_thresholds[risk_id] = (
40 |             float(best_threshold)
41 |         )
42 |
43 |     print(
44 |         risk_id,
45 |         "threshold:",
46 |         round(best_threshold, 2),
47 |         "F1:",
48 |         round(best_f1, 4)
49 |     )

```

CELL 12**Code cell 12**

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Defines probability-to-label conversion. For each event it includes every risk whose probability meets that risk's learned threshold, preserving multi-label behavior and allowing an empty result.

EXACT NOTEBOOK CODE

```
1 | # Convert probabilities into predictions
2 | def get_stage_b_predictions(
3 |     probability_matrix
4 | ):
5 |     all_predictions = []
6 |
7 |     for row_probabilities in probability_matrix:
8 |
9 |         row_predictions = [
10 |             risk_id
11 |             for risk_index, risk_id in enumerate(
12 |                 RISK_IDS
13 |             )
14 |             if row_probabilities[risk_index]
15 |                 >= stage_b_thresholds[risk_id]
16 |         ]
17 |
18 |         all_predictions.append(
19 |             row_predictions
20 |         )
21 |
22 |     return all_predictions
```

CELL 13**Code cell 13**

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Generates Stage B train and test prediction lists, counts empty predictions, and reports how many records received classifications. Training predictions are based on out-of-fold probabilities; test predictions come from the full-data model.

EXACT NOTEBOOK CODE

```
1 | # Generate Stage B predictions
2 | train_poc["stage_b_predictions"] = (
3 |     get_stage_b_predictions(
4 |         train_stage_b_probabilities
5 |     )
6 | )
7 |
8 | test_poc["stage_b_predictions"] = (
9 |     get_stage_b_predictions(
10 |         test_stage_b_probabilities
11 |     )
12 | )
13 |
14 |
15 | train_empty_predictions = (
16 |     train_poc["stage_b_predictions"].apply(
17 |         len
18 |     ) == 0
19 | ).sum()
20 |
21 | test_empty_predictions = (
22 |     test_poc["stage_b_predictions"].apply(
23 |         len
24 |     ) == 0
25 | ).sum()
26 |
27 |
28 | print(
29 |     "Stage B training classifications:",
30 |     len(train_poc)
31 | )
32 |
33 | print(
34 |     "Stage B test classifications:",
35 |     len(test_poc)
36 | )
37 |
38 | print(
```

```

39 |     "Training records without a confident prediction:",
40 |     train_empty_predictions
41 | )
42 |
43 | print(
44 |     "Test records without a confident prediction:",
45 |     test_empty_predictions
46 | )

```

CELL 14

Code cell 14

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Displays ten training records with IDs, headlines, true labels, and Stage B predictions for qualitative inspection.

EXACT NOTEBOOK CODE

```

1 | # Inspect predictions
2 | train_poc[
3 |     [
4 |         "id",
5 |         "Headline",
6 |         "true_risks",
7 |         "stage_b_predictions"
8 |     ]
9 | ].head(10)

```

CELL 15

Code cell 15

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Defines Stage B evaluation with the same six-label binarization and aggregate metrics used in Stage A, excluding coverage because coverage is reported separately in Cell 13.

EXACT NOTEBOOK CODE

```

1 | # Define the Stage B evaluation function
2 | RISK_IDS = list(
3 |     RISK_TAXONOMY.keys()

```

```

4 | )
5 |
6 |
7 | def evaluate_stage_b(dataframe, split_name):
8 |
9 |     label_binarizer = MultiLabelBinarizer(
10 |         classes=RISK_IDS
11 |     )
12 |
13 |     true_matrix = label_binarizer.fit_transform(
14 |         dataframe["true_risks"]
15 |     )
16 |
17 |     predicted_matrix = label_binarizer.transform(
18 |         dataframe["stage_b_predictions"]
19 |     )
20 |
21 |     print(
22 |         f"{split_name} Stage B classification report"
23 |     )
24 |
25 |     print(
26 |         classification_report(
27 |             true_matrix,
28 |             predicted_matrix,
29 |             target_names=[
30 |                 RISK_TAXONOMY[risk_id]["name"]
31 |                 for risk_id in RISK_IDS
32 |             ],
33 |             zero_division=0
34 |         )
35 |     )
36 |
37 |     micro_precision, micro_recall, micro_f1, _ = (
38 |         precision_recall_fscore_support(
39 |             true_matrix,
40 |             predicted_matrix,
41 |             average="micro",
42 |             zero_division=0

```

```

43 |     )
44 | )
45 |
46 | macro_precision, macro_recall, macro_f1, _ = (
47 |     precision_recall_fscore_support(
48 |         true_matrix,
49 |         predicted_matrix,
50 |         average="macro",
51 |         zero_division=0
52 |     )
53 | )
54 |
55 | exact_match = np.mean(
56 |     [
57 |         set(true_risks) == set(predicted_risks)
58 |         for true_risks, predicted_risks in zip(
59 |             dataframe["true_risks"],
60 |             dataframe["stage_b_predictions"]
61 |         )
62 |     ]
63 | )
64 |
65 | print(
66 |     "Micro precision:",
67 |     round(micro_precision, 4)
68 | )
69 |
70 | print(
71 |     "Micro recall:",
72 |     round(micro_recall, 4)
73 | )
74 |
75 | print(
76 |     "Micro F1:",
77 |     round(micro_f1, 4)
78 | )
79 |
80 | print(
81 |     "Macro precision:",

```

```

82 |         round(macro_precision, 4)
83 |     )
84 |
85 |     print(
86 |         "Macro recall:",
87 |         round(macro_recall, 4)
88 |     )
89 |
90 |     print(
91 |         "Macro F1:",
92 |         round(macro_f1, 4)
93 |     )
94 |
95 |     print(
96 |         "Exact multi-label match:",
97 |         round(exact_match, 4)
98 |     )
99 |
100 |     return {
101 |         "micro_precision": micro_precision,
102 |         "micro_recall": micro_recall,
103 |         "micro_f1": micro_f1,
104 |         "macro_precision": macro_precision,
105 |         "macro_recall": macro_recall,
106 |         "macro_f1": macro_f1,
107 |         "exact_match": exact_match
108 |     }

```

CELL 16

Code cell 16

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Runs Stage B evaluation on the out-of-fold training predictions and stores the aggregate metrics.

EXACT NOTEBOOK CODE

```
1 | # Evaluate training predictions
2 | train_stage_b_metrics = evaluate_stage_b(
3 |     train_poc,
4 |     "Training"
5 | )
```

CELL 17

Code cell 17

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Runs Stage B evaluation on held-out test predictions.

EXACT NOTEBOOK CODE

```
1 | # Evaluate test predictions
2 |
3 | test_stage_b_metrics = evaluate_stage_b(
4 |     test_poc,
5 |     "Test"
6 | )
```

CELL 18

Code cell 18

Execution count: 32

WHAT THIS CELL IMPLEMENTS

Combines the train and test metric dictionaries into a rounded comparison table.

EXACT NOTEBOOK CODE

```
1 | # Compare the two reports
2 | stage_b_metric_comparison = pd.DataFrame(
3 |     [
4 |         train_stage_b_metrics,
5 |         test_stage_b_metrics
6 |     ],
7 |     index=[
8 |         "Training",
9 |         "Test"
10 |    ]
11 | )
12 |
13 | stage_b_metric_comparison.round(4)
```

CELL 19

Code cell 19

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Merges ground truth and Stage B predictions with Stage A predictions by ID for both splits. `validate='one\_to\_one'` guards against duplicate IDs on either side.

EXACT NOTEBOOK CODE

```
1 | # Merge the Stage A and Stage B results using id.
2 | train_results = train_poc[
3 |     [
4 |         "id",
5 |         "true_risks",
6 |         "stage_b_predictions"
7 |     ]
8 | ].merge(
9 |     stage_a_train[
10 |         [
11 |             "id",
12 |             "stage_a_predictions"
13 |         ]
14 |     ],
15 |     on="id",
16 |     how="left",
```

```

17 |     validate="one_to_one"
18 | )
19 |
20 |
21 | test_results = test_poc[
22 |     [
23 |         "id",
24 |         "true_risks",
25 |         "stage_b_predictions"
26 |     ]
27 | ].merge(
28 |     stage_a_test[
29 |         [
30 |             "id",
31 |             "stage_a_predictions"
32 |         ]
33 |     ],
34 |     on="id",
35 |     how="left",
36 |     validate="one_to_one"
37 | )
38 |
39 |
40 | print(
41 |     "Merged training records:",
42 |     len(train_results)
43 | )
44 |
45 | print(
46 |     "Merged test records:",
47 |     len(test_results)
48 | )

```

WHAT THIS CELL IMPLEMENTS

Defines the Stage C router and applies it. Exact non-empty A/B agreement is accepted; an empty A with non-empty B accepts B; both empty, B empty while A is non-empty, or any non-empty disagreement is routed to Stage C with a reason code and no accepted prediction yet.

EXACT NOTEBOOK CODE

```

1 | # Decide whether each record should be sent to Stage C.
2 | def get_stage_c_decision(row):
3 |
4 |     stage_a_predictions = (
5 |         row["stage_a_predictions"]
6 |     )
7 |
8 |     stage_b_predictions = (
9 |         row["stage_b_predictions"]
10 |     )
11 |
12 |     stage_a_set = set(
13 |         stage_a_predictions
14 |     )
15 |
16 |     stage_b_set = set(
17 |         stage_b_predictions
18 |     )
19 |
20 |
21 |     # Accept an exact, non-empty agreement.
22 |     if (
23 |         stage_a_set
24 |         and stage_a_set == stage_b_set
25 |     ):
26 |         return pd.Series({
27 |             "route_to_stage_c": False,
28 |             "accepted_predictions": stage_a_predictions,
29 |             "stage_c_route_reason": (
30 |                 "stage_a_stage_b_agree"
31 |             )

```

```

32 |         })
33 |
34 |
35 |     # Use Stage B when Stage A has no prediction.
36 |     if (
37 |         not stage_a_set
38 |         and stage_b_set
39 |     ):
40 |         return pd.Series({
41 |             "route_to_stage_c": False,
42 |             "accepted_predictions": stage_b_predictions,
43 |             "stage_c_route_reason": (
44 |                 "stage_a_empty_stage_b_confident"
45 |             )
46 |         })
47 |
48 |
49 |     # Route every other case to Stage C.
50 |     if (
51 |         not stage_a_set
52 |         and not stage_b_set
53 |     ):
54 |         route_reason = "both_stages_empty"
55 |
56 |     elif (
57 |         stage_a_set
58 |         and not stage_b_set
59 |     ):
60 |         route_reason = "stage_b_empty"
61 |
62 |     else:
63 |         route_reason = "predictions_disagree"
64 |
65 |
66 |     return pd.Series({
67 |         "route_to_stage_c": True,
68 |         "accepted_predictions": [],
69 |         "stage_c_route_reason": route_reason
70 |     })

```

```

71 |
72 |
73 | train_routing = train_results.apply(
74 |     get_stage_c_decision,
75 |     axis=1
76 | )
77 |
78 | test_routing = test_results.apply(
79 |     get_stage_c_decision,
80 |     axis=1
81 | )
82 |
83 |
84 | train_results = pd.concat(
85 |     [
86 |         train_results,
87 |         train_routing
88 |     ],
89 |     axis=1
90 | )
91 |
92 | test_results = pd.concat(
93 |     [
94 |         test_results,
95 |         test_routing
96 |     ],
97 |     axis=1
98 | )

```

CELL 21

Code cell 21

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Defines a fair side-by-side comparison of Stage A and Stage B on the same merged rows, calculating micro metrics, macro F1, exact match, and non-empty coverage.

EXACT NOTEBOOK CODE

```

1 | # Compare Stage A and Stage B before Stage C.
2 | def compare_prediction_methods(dataframe):

```

```

3 |
4 |     true_matrix = label_binarizer.transform(
5 |         dataframe["true_risks"]
6 |     )
7 |
8 |     comparison_rows = []
9 |
10 |
11 |     prediction_columns = {
12 |         "Stage A": "stage_a_predictions",
13 |         "Stage B": "stage_b_predictions"
14 |     }
15 |
16 |
17 |     for method_name, column_name in (
18 |         prediction_columns.items()
19 |     ):
20 |
21 |         predicted_matrix = label_binarizer.transform(
22 |             dataframe[column_name]
23 |         )
24 |
25 |         micro_precision, micro_recall, micro_f1, _ = (
26 |             precision_recall_fscore_support(
27 |                 true_matrix,
28 |                 predicted_matrix,
29 |                 average="micro",
30 |                 zero_division=0
31 |             )
32 |         )
33 |
34 |         _, _, macro_f1, _ = (
35 |             precision_recall_fscore_support(
36 |                 true_matrix,
37 |                 predicted_matrix,
38 |                 average="macro",
39 |                 zero_division=0
40 |             )
41 |         )

```

```

42 |
43 |     exact_match = np.mean(
44 |         np.all(
45 |             true_matrix == predicted_matrix,
46 |             axis=1
47 |         )
48 |     )
49 |
50 |     coverage = dataframe[column_name].apply(
51 |         lambda predictions: len(predictions) > 0
52 |     ).mean()
53 |
54 |     comparison_rows.append({
55 |         "method": method_name,
56 |         "micro_precision": micro_precision,
57 |         "micro_recall": micro_recall,
58 |         "micro_f1": micro_f1,
59 |         "macro_f1": macro_f1,
60 |         "exact_match": exact_match,
61 |         "coverage": coverage
62 |     })
63 |
64 |
65 |     return pd.DataFrame(
66 |         comparison_rows
67 |     ).set_index(
68 |         "method"
69 |     )

```

CELL 22

Code cell 22

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Runs the A-versus-B comparison on training data.

EXACT NOTEBOOK CODE

```
1 | # Training comparison
2 | train_method_comparison = (
3 |     compare_prediction_methods(
4 |         train_results
5 |     )
6 | )
7 |
8 | train_method_comparison.round(4)
```

CELL 23

Code cell 23

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Runs the same method comparison on the test data.

EXACT NOTEBOOK CODE

```
1 | # Test comparison
2 | test_method_comparison = (
3 |     compare_prediction_methods(
4 |         test_results
5 |     )
6 | )
7 |
8 | test_method_comparison.round(4)
```

CELL 24

Code cell 24

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Prints how many train and test rows are routed to Stage C and breaks the counts down by route reason.

EXACT NOTEBOOK CODE

```
1 | # Check how many records will be processed by Stage C.
2 | print(
3 |     "Training records routed to Stage C:",
4 |     int(train_results["route_to_stage_c"].sum()),
5 |     "of",
```

```

6 |     len(train_results)
7 | )
8 |
9 | print(
10 |     train_results[
11 |         "stage_c_route_reason"
12 |     ].value_counts()
13 | )
14 |
15 |
16 | print(
17 |     "\nTest records routed to Stage C:",
18 |     int(test_results["route_to_stage_c"].sum()),
19 |     "of",
20 |     len(test_results)
21 | )
22 |
23 | print(
24 |     test_results[
25 |         "stage_c_route_reason"
26 |     ].value_counts()
27 | )

```

CELL 25

Code cell 25

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Converts each probability vector into a dictionary keyed by risk ID and rounds values to six decimals for readable, stable CSV export.

EXACT NOTEBOOK CODE

```

1 | # Prepare Stage B probabilities for export
2 | train_probability_records = [
3 |     {
4 |         risk_id: round(
5 |             float(row_probabilities[risk_index]),
6 |             6
7 |         )
8 |         for risk_index, risk_id in enumerate(

```

```

 9 |         RISK_IDS
10 |     )
11 | }
12 | for row_probabilities in (
13 |     train_stage_b_probabilities
14 | )
15 | ]
16 |
17 |
18 | test_probability_records = [
19 |     {
20 |         risk_id: round(
21 |             float(row_probabilities[risk_index]),
22 |             6
23 |         )
24 |         for risk_index, risk_id in enumerate(
25 |             RISK_IDS
26 |         )
27 |     }
28 |     for row_probabilities in (
29 |         test_stage_b_probabilities
30 |     )
31 | ]

```

CELL 26

Code cell 26

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Serializes Stage B prediction lists and probability dictionaries as JSON and exports ID-keyed train and test prediction CSVs.

EXACT NOTEBOOK CODE

```

1 | # Export Stage B predictions
2 | OUTPUT_DIR.mkdir(
3 |     parents=True,
4 |     exist_ok=True
5 | )
6 |
7 |

```

```

8 | stage_b_train_output = pd.DataFrame({
9 |     "id": train_poc["id"],
10 |     "stage_b_predictions": (
11 |         train_poc["stage_b_predictions"].apply(
12 |             lambda predictions: json.dumps(
13 |                 predictions,
14 |                 ensure_ascii=False
15 |             )
16 |         )
17 |     ),
18 |     "stage_b_probabilities": [
19 |         json.dumps(
20 |             probabilities,
21 |             ensure_ascii=False
22 |         )
23 |         for probabilities in (
24 |             train_probability_records
25 |         )
26 |     ]
27 | })
28 |
29 |
30 | stage_b_test_output = pd.DataFrame({
31 |     "id": test_poc["id"],
32 |     "stage_b_predictions": (
33 |         test_poc["stage_b_predictions"].apply(
34 |             lambda predictions: json.dumps(
35 |                 predictions,
36 |                 ensure_ascii=False
37 |             )
38 |         )
39 |     ),
40 |     "stage_b_probabilities": [
41 |         json.dumps(
42 |             probabilities,
43 |             ensure_ascii=False
44 |         )
45 |         for probabilities in (
46 |             test_probability_records

```

```

47 |         )
48 |     ]
49 | })
50 |
51 |
52 | stage_b_train_output.to_csv(
53 |     OUTPUT_DIR / "stage_b_train_predictions.csv",
54 |     index=False
55 | )
56 |
57 | stage_b_test_output.to_csv(
58 |     OUTPUT_DIR / "stage_b_test_predictions.csv",
59 |     index=False
60 | )
61 |
62 |
63 | print(
64 |     "Saved:",
65 |     OUTPUT_DIR / "stage_b_train_predictions.csv"
66 | )
67 |
68 | print(
69 |     "Saved:",
70 |     OUTPUT_DIR / "stage_b_test_predictions.csv"
71 | )

```

CELL 27

Code cell 27

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Builds the full Stage C handoff tables by merging text, Stage A/B predictions, routing status, accepted predictions, route reason, and probability dictionaries by ID.

EXACT NOTEBOOK CODE

```

1 | # Prepare probability records with their matching IDs.
2 | train_probability_data = pd.DataFrame({
3 |     "id": train_poc["id"],
4 |     "stage_b_probabilities": (
5 |         train_probability_records

```

```

6 |     )
7 | })
8 |
9 | test_probability_data = pd.DataFrame({
10 |     "id": test_poc["id"],
11 |     "stage_b_probabilities": (
12 |         test_probability_records
13 |     )
14 | })
15 |
16 |
17 | # Create the complete Stage C input data.
18 | train_stage_c_input = train_poc[
19 |     [
20 |         "id",
21 |         "input_text"
22 |     ]
23 | ].merge(
24 |     train_results[
25 |         [
26 |             "id",
27 |             "stage_a_predictions",
28 |             "stage_b_predictions",
29 |             "route_to_stage_c",
30 |             "accepted_predictions",
31 |             "stage_c_route_reason"
32 |         ]
33 |     ],
34 |     on="id",
35 |     how="left",
36 |     validate="one_to_one"
37 | ).merge(
38 |     train_probability_data,
39 |     on="id",
40 |     how="left",
41 |     validate="one_to_one"
42 | )
43 |
44 |

```

```

45 | test_stage_c_input = test_poc[
46 |     [
47 |         "id",
48 |         "input_text"
49 |     ]
50 | ].merge(
51 |     test_results[
52 |         [
53 |             "id",
54 |             "stage_a_predictions",
55 |             "stage_b_predictions",
56 |             "route_to_stage_c",
57 |             "accepted_predictions",
58 |             "stage_c_route_reason"
59 |         ]
60 |     ],
61 |     on="id",
62 |     how="left",
63 |     validate="one_to_one"
64 | ).merge(
65 |     test_probability_data,
66 |     on="id",
67 |     how="left",
68 |     validate="one_to_one"
69 | )

```

CELL 28

Code cell 28

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

JSON-serializes list/dictionary columns and exports `stage\_c\_train\_input.csv` and `stage\_c\_test\_input.csv`. These files contain both already-resolved rows and cases that Stage C must adjudicate.

EXACT NOTEBOOK CODE

```

1 | # Convert lists and dictionaries into JSON text for CSV.
2 | json_columns = [
3 |     "stage_a_predictions",
4 |     "stage_b_predictions",
5 |     "accepted_predictions",

```

```

6 |     "stage_b_probabilities"
7 | ]
8 |
9 |
10 | for column_name in json_columns:
11 |
12 |     train_stage_c_input[column_name] = (
13 |         train_stage_c_input[column_name].apply(
14 |             lambda value: json.dumps(
15 |                 value,
16 |                 ensure_ascii=False
17 |             )
18 |         )
19 |     )
20 |
21 |     test_stage_c_input[column_name] = (
22 |         test_stage_c_input[column_name].apply(
23 |             lambda value: json.dumps(
24 |                 value,
25 |                 ensure_ascii=False
26 |             )
27 |         )
28 |     )
29 |
30 |
31 | train_stage_c_input.to_csv(
32 |     OUTPUT_DIR / "stage_c_train_input.csv",
33 |     index=False
34 | )
35 |
36 | test_stage_c_input.to_csv(
37 |     OUTPUT_DIR / "stage_c_test_input.csv",
38 |     index=False
39 | )
40 |
41 |
42 | print(
43 |     "Saved:",
44 |     OUTPUT_DIR / "stage_c_train_input.csv"

```

```
45 | )  
46 |  
47 | print(  
48 |     "Saved:",  
49 |     OUTPUT_DIR / "stage_c_test_input.csv"  
50 | )
```

Stage C - Retrieval-Augmented LLM

Nearest-example retrieval, structured adjudication, fallback, and final export

SOURCE notebooks/04_stage_c_rag_llm.ipynb

19 CODE CELLS

CELL 1

Code cell 1

Execution count: 1

WHAT THIS CELL IMPLEMENTS

Imports serialization, path, numerical, and tabular utilities; resolves the project root; and defines data/output directories.

EXACT NOTEBOOK CODE

```
1 | import ast
2 | import json
3 | from pathlib import Path
4 |
5 | import numpy as np
6 | import pandas as pd
7 |
8 |
9 | CURRENT_DIR = Path.cwd()
10 |
11 | if CURRENT_DIR.name == "notebooks":
12 |     PROJECT_ROOT = CURRENT_DIR.parent
13 | else:
14 |     PROJECT_ROOT = CURRENT_DIR
15 |
16 |
17 | DATA_DIR = PROJECT_ROOT / "data"
18 | OUTPUT_DIR = PROJECT_ROOT / "outputs"
```

CELL 2**Code cell 2**

Execution count: 2

WHAT THIS CELL IMPLEMENTS

Loads training IDs, text, and truth plus both Stage C handoff tables. It restores training truth lists with `ast.literal\_eval` for retrieval labels and evaluation.

EXACT NOTEBOOK CODE

```
1 | train_poc = pd.read_csv(  
2 |     DATA_DIR / "train_poc.csv",  
3 |     usecols=["id", "input_text", "true_risks"]  
4 | )  
5 |  
6 | train_input = pd.read_csv(  
7 |     OUTPUT_DIR / "stage_c_train_input.csv"  
8 | )  
9 |  
10 | test_input = pd.read_csv(  
11 |     OUTPUT_DIR / "stage_c_test_input.csv"  
12 | )  
13 |  
14 |  
15 | train_poc["true_risks"] = (  
16 |     train_poc["true_risks"].apply(ast.literal_eval)  
17 | )
```

CELL 3**Code cell 3**

Execution count: 3

WHAT THIS CELL IMPLEMENTS

Deserializes four JSON columns in both handoff tables and normalizes the CSV routing flag to a real Boolean using a case-insensitive comparison to `true`.

EXACT NOTEBOOK CODE

```
1 | json_columns = [  
2 |     "stage_a_predictions",  
3 |     "stage_b_predictions",  
4 |     "accepted_predictions",  
5 |     "stage_b_probabilities"  
6 | ]  
7 |  
8 |  
9 | for dataframe in [train_input, test_input]:  
10 |  
11 |     for column in json_columns:  
12 |         dataframe[column] = dataframe[column].apply(  
13 |             json.loads  
14 |         )  
15 |  
16 |     dataframe["route_to_stage_c"] = (  
17 |         dataframe["route_to_stage_c"]  
18 |         .astype(str)  
19 |         .str.lower()  
20 |         .eq("true")  
21 |     )
```

CELL 4

Code cell 4

Execution count: 4

WHAT THIS CELL IMPLEMENTS

Loads the saved NPZ embeddings and IDs, verifies that training embedding order exactly matches `train\_poc`, and prints the embedding model. Test-ID order is loaded but not explicitly asserted against `test\_input` here; later ID-position lookup controls query selection.

EXACT NOTEBOOK CODE

```
1 | saved_embeddings = np.load(  
2 |     OUTPUT_DIR / "stage_b_embeddings.npz",  
3 |     allow_pickle=False  
4 | )  
5 |  
6 | train_ids = saved_embeddings["train_ids"]  
7 | test_ids = saved_embeddings["test_ids"]  
8 |  
9 | train_embeddings = saved_embeddings["train_embeddings"]  
10 | test_embeddings = saved_embeddings["test_embeddings"]  
11 |  
12 |  
13 | assert np.array_equal(  
14 |     train_ids,  
15 |     train_poc["id"].to_numpy()  
16 | )  
17 |  
18 | print(  
19 |     "Embedding model:",  
20 |     saved_embeddings["model_name"].item()  
21 | )
```

CELL 5

Code cell 5

Execution count: 5

WHAT THIS CELL IMPLEMENTS

Filters to routed cases. For train-side LLM evaluation it draws a reproducible, capped 100-case sample stratified by three routing reasons (45 disagreements, 40 both-empty, 15 B-empty); every routed test case is retained. Existing artifacts show 100 train-sample and 194 routed test rows.

EXACT NOTEBOOK CODE

```
1 | routed_train = train_input[  
2 |     train_input["route_to_stage_c"]  
3 | ].copy()  
4 |  
5 | routed_test = test_input[  
6 |     test_input["route_to_stage_c"]  
7 | ].copy()
```

```

8 |
9 |
10 | sample_sizes = {
11 |     "predictions_disagree": 45,
12 |     "both_stages_empty": 40,
13 |     "stage_b_empty": 15
14 | }
15 |
16 |
17 | training_samples = []
18 |
19 | for reason, sample_size in sample_sizes.items():
20 |
21 |     group = routed_train[
22 |         routed_train["stage_c_route_reason"] == reason
23 |     ]
24 |
25 |     training_samples.append(
26 |         group.sample(
27 |             n=min(sample_size, len(group)),
28 |             random_state=42
29 |         )
30 |     )
31 |
32 |
33 | train_llm_input = pd.concat(
34 |     training_samples,
35 |     ignore_index=True
36 | )
37 |
38 | test_llm_input = routed_test.reset_index(
39 |     drop=True
40 | )
41 |
42 |
43 | print("Training LLM records:", len(train_llm_input))
44 | print("Test LLM records:", len(test_llm_input))

```

CELL 6

Code cell 6

Execution count: 6

WHAT THIS CELL IMPLEMENTS

Builds ID-to-embedding-position dictionaries and gathers the embedding rows corresponding to the sampled train queries and routed test queries.

EXACT NOTEBOOK CODE

```
1 | train_position = {
2 |     event_id: position
3 |     for position, event_id in enumerate(train_ids)
4 | }
5 |
6 | test_position = {
7 |     event_id: position
8 |     for position, event_id in enumerate(test_ids)
9 | }
10 |
11 |
12 | train_query_embeddings = train_embeddings[
13 |     [
14 |         train_position[event_id]
15 |         for event_id in train_llm_input["id"]
16 |     ]
17 | ]
18 |
19 | test_query_embeddings = test_embeddings[
20 |     [
21 |         test_position[event_id]
22 |         for event_id in test_llm_input["id"]
23 |     ]
24 | ]
```

WHAT THIS CELL IMPLEMENTS

Defines exact nearest-neighbor retrieval over normalized embeddings. A query-by-training dot product yields cosine similarities, training queries can exclude themselves, the top three positions are selected, and each retrieved example includes ID, text, true labels, and similarity.

EXACT NOTEBOOK CODE

```
1 | train_lookup = train_poc.set_index("id")
2 |
3 |
4 | def add_retrieved_examples(
5 |     queries,
6 |     query_embeddings,
7 |     exclude_self=False,
8 |     k=3
9 | ):
10 |
11 |     similarities = (
12 |         query_embeddings
13 |         @ train_embeddings.T
14 |     )
15 |
16 |     if exclude_self:
17 |
18 |         for row_number, event_id in enumerate(
19 |             queries["id"]
20 |         ):
21 |             similarities[
22 |                 row_number,
23 |                 train_position[event_id]
24 |             ] = -np.inf
25 |
26 |     nearest_positions = np.argsort(
27 |         similarities,
28 |         axis=1
29 |     )[:, -k:][:, :-1]
30 |
31 |     output = queries.copy()
```

```

32 |
33 |     output["retrieved_examples"] = [
34 |         [
35 |             {
36 |                 "id": int(train_ids[position]),
37 |                 "text": train_lookup.at[
38 |                     int(train_ids[position]),
39 |                     "input_text"
40 |                 ],
41 |                 "true_risks": train_lookup.at[
42 |                     int(train_ids[position]),
43 |                     "true_risks"
44 |                 ],
45 |                 "similarity": round(
46 |                     float(similarities[row_number, position]),
47 |                     4
48 |                 )
49 |             }
50 |             for position in positions
51 |         ]
52 |         for row_number, positions in enumerate(
53 |             nearest_positions
54 |         )
55 |     ]
56 |
57 |     return output

```

CELL 8

Code cell 8

Execution count: 8

WHAT THIS CELL IMPLEMENTS

Adds three retrieved labeled examples to every train and test LLM case. Self-retrieval is disabled for training evaluation and allowed for test-to-training retrieval; a preview is displayed.

EXACT NOTEBOOK CODE

```
1 | train_llm_input = add_retrieved_examples(  
2 |     train_llm_input,  
3 |     train_query_embeddings,  
4 |     exclude_self=True  
5 | )  
6 |  
7 | test_llm_input = add_retrieved_examples(  
8 |     test_llm_input,  
9 |     test_query_embeddings  
10 | )  
11 |  
12 |  
13 | train_llm_input[  
14 |     [  
15 |         "id",  
16 |         "stage_c_route_reason",  
17 |         "retrieved_examples"  
18 |     ]  
19 | ].head()
```

CELL 9

Code cell 9

Execution count: 9

WHAT THIS CELL IMPLEMENTS

Installs the OpenAI Python package inside the active notebook kernel environment. This is an environment-changing notebook magic and may require a kernel restart.

EXACT NOTEBOOK CODE

```
1 | %pip install -q openai
```

CELL 10

Code cell 10

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Reads `OPENAI\_API\_KEY` from the environment or securely prompts for it, constructs an OpenAI client, and selects `gpt-4.1-mini`. Running later cells can incur API usage and requires network access.

EXACT NOTEBOOK CODE

```
1 | import os
2 | import getpass
3 |
4 | from openai import OpenAI
5 |
6 |
7 | api_key = os.getenv("OPENAI_API_KEY")
8 |
9 | if not api_key:
10 |     api_key = getpass.getpass(
11 |         "Enter OpenAI API key: "
12 |     )
13 |
14 | client = OpenAI(api_key=api_key)
15 |
16 | LLM_MODEL = "gpt-4.1-mini"
```

CELL 11

Code cell 11

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Defines compact risk descriptions, fixes the allowed risk IDs, and creates a strict JSON Schema requiring exactly `predicted\_risks` and `reason`, with prediction values restricted to the six taxonomy IDs.

EXACT NOTEBOOK CODE

```
1 | # Taxonomy and output format
2 | RISK_TAXONOMY = {
3 |     "weather_disruption":
4 |         "Disruption caused by severe weather.",
5 |
6 |     "natural_disaster":
7 |         "Disruption caused by earthquakes, tsunamis, volcanoes or landslides.",
8 |
9 |     "port_operational_disruption":
10 |         "Port congestion, delays, backlogs or reduced operational capacity.",
11 |
12 |     "port_closure":
13 |         "Full or partial closure or suspension of a port or terminal.",
```

```

14 |
15 |     "labor_strike_disruption":
16 |         "Disruption caused by strikes or other labor action.",
17 |
18 |     "maritime_security_navigation_disruption":
19 |         "Piracy, maritime security threats, navigation restrictions or waterway disruption."
20 | }
21 |
22 |
23 | RISK_IDS = list(RISK_TAXONOMY)
24 |
25 |
26 | DECISION_FORMAT = {
27 |     "type": "json_schema",
28 |     "name": "stage_c_decision",
29 |     "strict": True,
30 |     "schema": {
31 |         "type": "object",
32 |         "properties": {
33 |             "predicted_risks": {
34 |                 "type": "array",
35 |                 "items": {
36 |                     "type": "string",
37 |                     "enum": RISK_IDS
38 |                 }
39 |             },
40 |             "reason": {
41 |                 "type": "string"
42 |             }
43 |         },
44 |         "required": [
45 |             "predicted_risks",
46 |             "reason"
47 |         ],
48 |         "additionalProperties": False
49 |     }
50 | }

```

WHAT THIS CELL IMPLEMENTS

Creates the adjudicator instructions, a deterministic non-LLM fallback, and the per-row LLM call. Each case contains target text, A/B predictions, B probabilities, route reason, and retrieved examples; the request uses structured output, temperature 0, no storage, two attempts, de-duplicates labels, and falls back to B then A after repeated errors.

EXACT NOTEBOOK CODE

```

1 | # Prompt, adjudication and fallback
2 | INSTRUCTIONS = f"""
3 | You are the final adjudicator for a multi-label risk classifier.
4 |
5 | Risk taxonomy:
6 | {json.dumps(RISK_TAXONOMY, indent=2)}
7 |
8 | Use the event text as the main evidence.
9 | Stage A, Stage B and the retrieved examples are supporting evidence.
10 | Select only risks directly supported by the target event.
11 | Keep the reason brief.
12 | """
13 |
14 |
15 | def simple_fallback(row):
16 |
17 |     if row["stage_b_predictions"]:
18 |         return row["stage_b_predictions"]
19 |
20 |     if row["stage_a_predictions"]:
21 |         return row["stage_a_predictions"]
22 |
23 |     return []
24 |
25 |
26 | def adjudicate(row):
27 |
28 |     case = {
29 |         "event_text": row["input_text"],
30 |         "stage_a_predictions":

```

```

31 |         row["stage_a_predictions"],
32 |         "stage_b_predictions":
33 |         row["stage_b_predictions"],
34 |         "stage_b_probabilities":
35 |         row["stage_b_probabilities"],
36 |         "route_reason":
37 |         row["stage_c_route_reason"],
38 |         "retrieved_examples":
39 |         row["retrieved_examples"]
40 |     }
41 |
42 |     last_error = ""
43 |
44 |     for _ in range(2):
45 |
46 |         try:
47 |             response = client.responses.create(
48 |                 model=LLM_MODEL,
49 |                 instructions=INSTRUCTIONS,
50 |                 input=json.dumps(
51 |                     case,
52 |                     ensure_ascii=False
53 |                 ),
54 |                 text={
55 |                     "format": DECISION_FORMAT
56 |                 },
57 |                 temperature=0,
58 |                 max_output_tokens=200,
59 |                 store=False
60 |             )
61 |
62 |             result = json.loads(
63 |                 response.output_text
64 |             )
65 |
66 |             predictions = list(
67 |                 dict.fromkeys(
68 |                     result["predicted_risks"]
69 |                 )

```

```

70 |         )
71 |
72 |         return (
73 |             predictions,
74 |             result["reason"],
75 |             False
76 |         )
77 |
78 |     except Exception as error:
79 |         last_error = str(error)
80 |
81 |     return (
82 |         simple_fallback(row),
83 |         f"Fallback used: {last_error[:150]}",
84 |         True
85 |     )

```

CELL 13

Code cell 13

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Defines sequential batch adjudication. It iterates through rows, calls the adjudicator, prints progress every ten cases, and writes predictions, reason, and fallback status into three new columns.

EXACT NOTEBOOK CODE

```

1 | # Run adjudication
2 | def run_adjudication(dataframe):
3 |
4 |     output = dataframe.copy().reset_index(
5 |         drop=True
6 |     )
7 |
8 |     results = []
9 |
10 |    for number, (_, row) in enumerate(
11 |        output.iterrows(),
12 |        start=1
13 |    ):
14 |

```

```

15 |         results.append(
16 |             adjudicate(row)
17 |         )
18 |
19 |         if number % 10 == 0:
20 |             print(
21 |                 f"Completed {number} of {len(output)}"
22 |             )
23 |
24 |     output[
25 |         [
26 |             "stage_c_predictions",
27 |             "stage_c_reason",
28 |             "fallback_used"
29 |         ]
30 |     ] = pd.DataFrame(
31 |         results,
32 |         index=output.index
33 |     )
34 |
35 |     return output

```

CELL 14

Code cell 14

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Defines evaluation for any prediction column, printing a per-risk classification report and returning micro F1, macro F1, and exact multi-label set accuracy.

EXACT NOTEBOOK CODE

```

1 | # Evaluate the training sample
2 | from sklearn.metrics import classification_report
3 | from sklearn.metrics import f1_score
4 | from sklearn.preprocessing import MultiLabelBinarizer
5 |
6 |
7 | def evaluate_predictions(
8 |     dataframe,
9 |     prediction_column

```

```

10 | ):
11 |
12 |     label_binarizer = MultiLabelBinarizer(
13 |         classes=RISK_IDS
14 |     )
15 |
16 |     true_matrix = label_binarizer.fit_transform(
17 |         dataframe["true_risks"]
18 |     )
19 |
20 |     predicted_matrix = label_binarizer.transform(
21 |         dataframe[prediction_column]
22 |     )
23 |
24 |     print(
25 |         classification_report(
26 |             true_matrix,
27 |             predicted_matrix,
28 |             target_names=RISK_IDS,
29 |             zero_division=0
30 |         )
31 |     )
32 |
33 |     return {
34 |         "micro_f1": f1_score(
35 |             true_matrix,
36 |             predicted_matrix,
37 |             average="micro",
38 |             zero_division=0
39 |         ),
40 |         "macro_f1": f1_score(
41 |             true_matrix,
42 |             predicted_matrix,
43 |             average="macro",
44 |             zero_division=0
45 |         ),
46 |         "exact_match": np.mean(
47 |             np.all(
48 |                 true_matrix == predicted_matrix,

```

```
49 |         axis=1
50 |     )
51 | )
52 | }
```

CELL 15

Code cell 15

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Merges true labels into the sampled training cases and attempts to evaluate `stage\_c\_predictions`. In the current notebook order, `run\_adjudication(train\_llm\_input)` is never called first, so this column does not exist and the cell will fail unless the missing training-adjudication call was executed elsewhere interactively.

EXACT NOTEBOOK CODE

```
1 | train_evaluation = train_llm_input.merge(
2 |     train_poc[
3 |         ["id", "true_risks"]
4 |     ],
5 |     on="id",
6 |     how="left",
7 |     validate="one_to_one"
8 | )
9 |
10 |
11 | train_sample_metrics = evaluate_predictions(
12 |     train_evaluation,
13 |     "stage_c_predictions"
14 | )
15 |
16 | train_sample_metrics
```

CELL 16

Code cell 16

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Runs LLM adjudication on all routed test cases, producing predictions, explanations, and fallback indicators.

EXACT NOTEBOOK CODE

```
1 | # Run the test adjudication
2 | test_llm_input = run_adjudication(
3 |     test_llm_input
4 | )
```

CELL 17

Code cell 17

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Extracts retrieved training IDs for auditability, merges Stage C results back into every test row, selects Stage C output for routed rows and previously accepted A/B output for non-routed rows, and normalizes missing fallback flags to false.

EXACT NOTEBOOK CODE

```
1 | # Build final test predictions
2 |
3 | test_llm_input["retrieved_ids"] = (
4 |     test_llm_input["retrieved_examples"].apply(
5 |         lambda examples: [
6 |             example["id"]
7 |             for example in examples
8 |         ]
9 |     )
10 | )
11 |
12 |
13 | stage_c_results = test_llm_input[
14 |     [
15 |         "id",
16 |         "stage_c_predictions",
17 |         "stage_c_reason",
18 |         "fallback_used",
19 |         "retrieved_ids"
20 |     ]
21 | ]
22 |
23 |
24 | test_final = test_input.merge(
```

```

25 |     stage_c_results,
26 |     on="id",
27 |     how="left",
28 |     validate="one_to_one"
29 | )
30 |
31 |
32 | test_final["final_predictions"] = [
33 |     stage_c_predictions
34 |     if route_to_stage_c
35 |     else accepted_predictions
36 |
37 |     for route_to_stage_c,
38 |         stage_c_predictions,
39 |         accepted_predictions
40 |     in zip(
41 |         test_final["route_to_stage_c"],
42 |         test_final["stage_c_predictions"],
43 |         test_final["accepted_predictions"]
44 |     )
45 | ]
46 |
47 |
48 | test_final["fallback_used"] = (
49 |     test_final["fallback_used"]
50 |     .fillna(False)
51 |     .astype(bool)
52 | )

```

CELL 18

Code cell 18

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Loads held-out truth, merges it with final pipeline results, and evaluates both the full test pipeline and the routed-only subset.

EXACT NOTEBOOK CODE

```

1 | # Final evaluation and export
2 | test_truth = pd.read_csv(

```

```

3 |     DATA_DIR / "test_poc.csv",
4 |     usecols=["id", "true_risks"]
5 | )
6 |
7 | test_truth["true_risks"] = (
8 |     test_truth["true_risks"].apply(
9 |         ast.literal_eval
10 |     )
11 | )
12 |
13 |
14 | test_evaluation = test_final.merge(
15 |     test_truth,
16 |     on="id",
17 |     how="left",
18 |     validate="one_to_one"
19 | )
20 |
21 |
22 | full_test_metrics = evaluate_predictions(
23 |     test_evaluation,
24 |     "final_predictions"
25 | )
26 |
27 | routed_test_metrics = evaluate_predictions(
28 |     test_evaluation[
29 |         test_evaluation["route_to_stage_c"]
30 |     ],
31 |     "final_predictions"
32 | )

```

CELL 19

Code cell 19

Execution count: not recorded

WHAT THIS CELL IMPLEMENTS

Creates a three-row metrics table, JSON-serializes nested export fields, writes the training-sample predictions, final test predictions, and metrics CSVs, then prints completion. This cell depends on the missing train adjudication noted in Cell 15 being resolved in the interactive state.

EXACT NOTEBOOK CODE

```
1 | metrics = pd.DataFrame(  
2 |     [  
3 |         train_sample_metrics,  
4 |         full_test_metrics,  
5 |         routed_test_metrics  
6 |     ],  
7 |     index=[  
8 |         "training_sample",  
9 |         "full_test_pipeline",  
10 |         "routed_test_only"  
11 |     ]  
12 | )  
13 |  
14 |  
15 | test_export = test_final.copy()  
16 |  
17 | json_columns = [  
18 |     "stage_a_predictions",  
19 |     "stage_b_predictions",  
20 |     "stage_b_probabilities",  
21 |     "accepted_predictions",  
22 |     "stage_c_predictions",  
23 |     "retrieved_ids",  
24 |     "final_predictions"  
25 | ]  
26 |  
27 |  
28 | for column in json_columns:  
29 |  
30 |     test_export[column] = test_export[column].apply(  
31 |         lambda value: json.dumps(  
32 |             value,  
33 |             ensure_ascii=False  
34 |         )  
35 |         if isinstance(value, (list, dict))  
36 |         else value  
37 |     )  
38 |
```

```
39 |  
40 | train_evaluation.to_csv(  
41 |     OUTPUT_DIR /  
42 |     "stage_c_train_sample_predictions.csv",  
43 |     index=False  
44 | )  
45 |  
46 | test_export.to_csv(  
47 |     OUTPUT_DIR /  
48 |     "stage_c_test_predictions.csv",  
49 |     index=False  
50 | )  
51 |  
52 | metrics.to_csv(  
53 |     OUTPUT_DIR / "stage_c_metrics.csv"  
54 | )  
55 |  
56 |  
57 | print("Stage C outputs saved.")
```

One-sentence project summary

A cost-aware, auditable multi-label cascade classifies clear maritime risks with deterministic rules, captures semantic variation with embedding-based classifiers, and escalates only unresolved disagreements to retrieval-augmented LLM adjudication.